VIT®
BHOPAL
www.vitbhopal.ac.in

Module-3
DESIGN CONCEPTS AND TESTING

Module-3

- **The Design Concepts**
  - The Design Model
  - Architectural Design
  - Structured Analysis (DFD model)
  - Structured Design (Structural Chart)
  - User Interface Design: Interface Analysis
  - Interface Design Steps
  - Requirements Modelling

- **Software Testing Fundamentals**
  - Black Box Testing
  - White Box Testing
  - Unit Testing
  - Integration Testing
  - Validation testing
  - System testing
  - Art of debugging

The Software Design Process

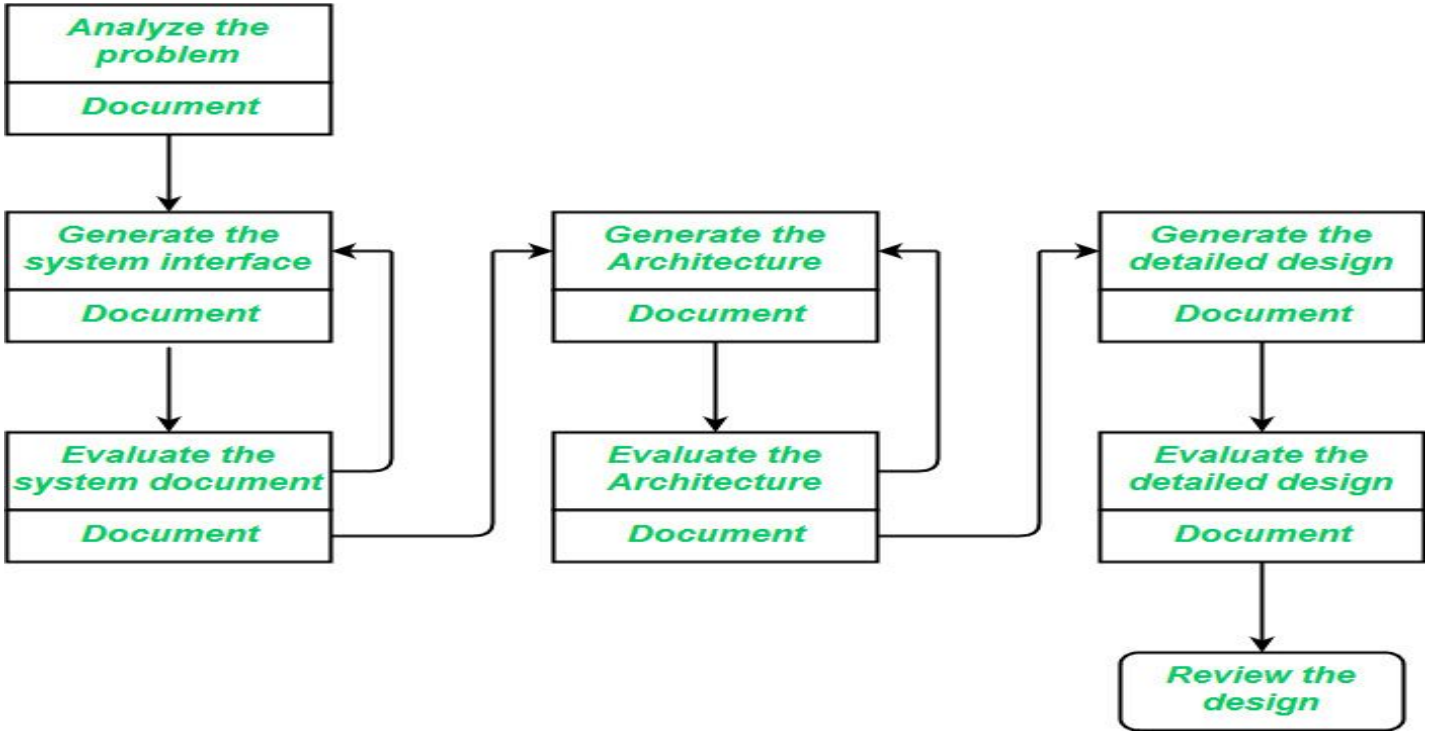
- The design phase of software development deals with transforming the customer requirements as described in the SRS documents into a form implementable using a programming language.
- Software design is an iterative process through which **requirements are translated into a "blueprint"** for constructing the software.
- Initially, design is represented at a **high level of abstraction** at a level that can be directly traced to the specific system objective and more detailed data, functional, and behavioral requirements.

The Software Design Process

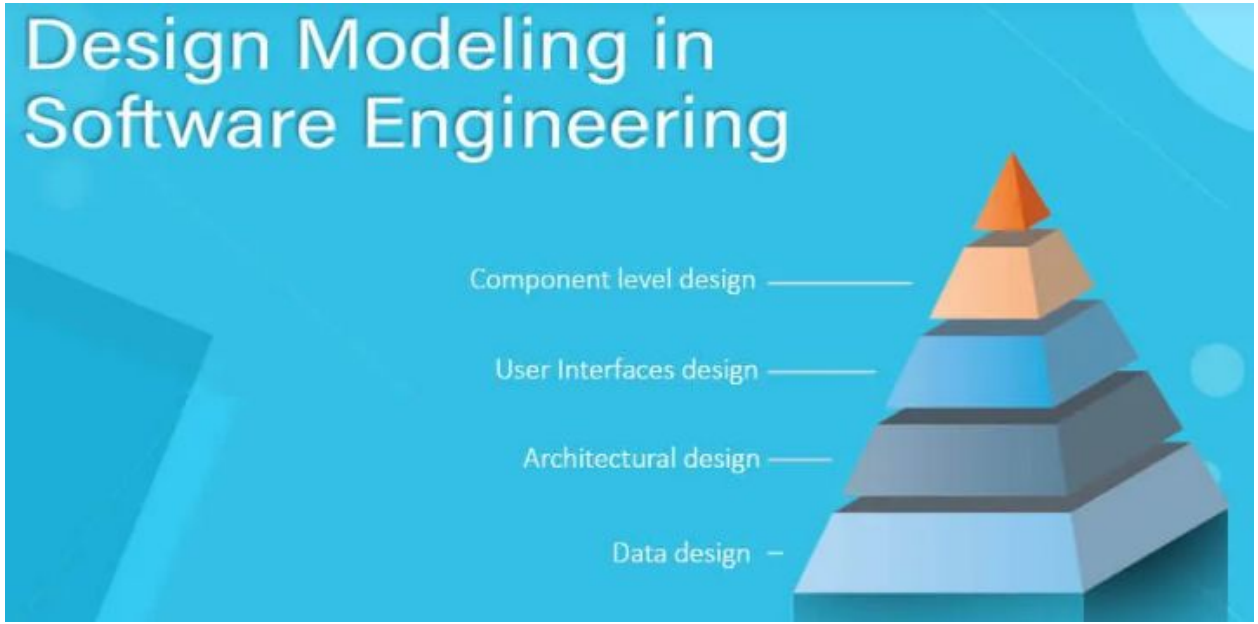
Elements of a System:

1. **Architecture –** This is the conceptual model that defines the structure, behavior, and views of a system. We can use flowcharts to represent and illustrate the architecture.
2. **Modules –** These are components that handle one specific task in a system. A combination of the modules makes up the system.
3. **Components –** This provides a particular function or group of related functions. They are made up of modules.
4. **Interfaces –** This is the shared boundary across which the components of a system exchange information and relate.
5. **Data –** This is the management of the information and data flow.

The Software Design Process



Design Modelling in Software Engineering



Design Modelling in Software Engineering

- Designing a model is an important phase and is a multi-process that represents the data structure, program structure, interface characteristic, and procedural details.
- It is mainly classified into four categories –
  - **Data design**
  - **Architectural design**
  - **Interface design, and**
  - **Component-level design.**

Design Modelling in Software Engineering

Data design:

- It represents the data objects and their interrelationship in an entity-relationship diagram.
- Entity-relationship consists of information required for each entity or data object, as well as showing the relationship between these objects.
- It shows the structure of the data in terms of the tables.
- It shows three types of relationships – one-to-one, one-to-many, and many-to-many.

Design Modelling in Software Engineering

Architectural design:

- It defines the relationship between major structural elements of the software.
- It is about decomposing the system into interacting components.
- It is expressed as a block diagram defining an overview of the system structure – features of the components and how these components communicate with each other to share data.
- It defines the structure and properties of the components that are involved in the system and also the inter-relationship among these components.

Design Modelling in Software Engineering

User Interfaces design:

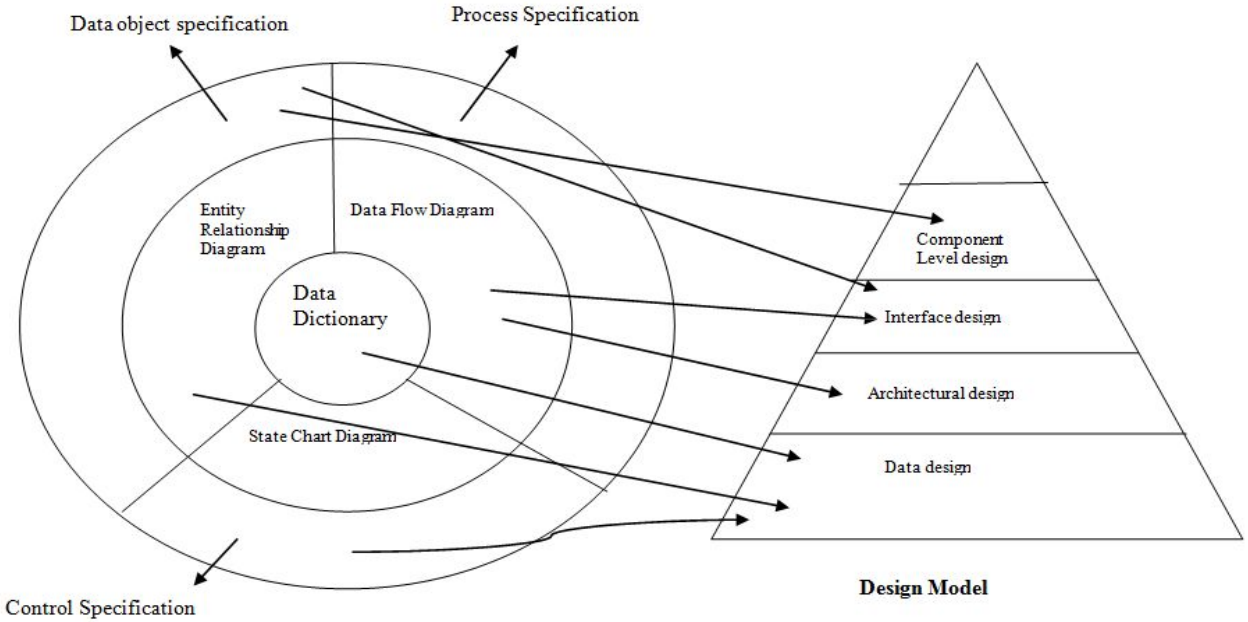
- It represents how the Software communicates with the user i.e. the behavior of the system.
- It refers to the product where users interact with controls or displays of the product.
- For example, Military, vehicles, aircraft, audio equipment, computer peripherals are the areas where user interface design is implemented.
- UI design becomes efficient only after performing usability testing. This is done to test what works and what does not work as expected. Only after making the repair, the product is said to have an optimized interface.

Design Modelling in Software Engineering

Component level design:

- It transforms the structural elements of the software architecture into a procedural description of software components.
- It is a perfect way to share a large amount of data.
- Components need not be concerned with how data is managed at a centralized level i.e. components need not worry about issues like backup and security of the data.

Design Modelling in Software Engineering



Design Concepts and Principles

Design Principles: -

- Software design is both a process and a model.
- The design process is a sequence of steps that enable the designer to describe all aspects of the software to be built.
- The design model is the equivalent of an architect’s plans for a house. It begins by representing the totality of the thing to be built and slowly refines the thing to provide guidance for constructing each detail.
- Similarly, the design model that is created for software provides a variety of different views of the computer software. Basic design principles enable the software engineer to navigate the design process.

Design Concepts and Principles

Principles for software design, which have been adapted and extended in the following list:

- The design should be traceable to the analysis model.
- The design should not reinvent the wheel. (it should not waste time or effort in creating things that already exist.)
- The structure of the software design should mimic the structure of the problem domain.
- The design should exhibit uniformity and integration.
- The design should be structured to accommodate change.
- The design should be assessed for quality as it is being created, not after the fact.
- The design should be reviewed to minimize conceptual (semantic) errors.

Design Concepts and Principles

Design concepts:

- **Abstraction:** Abstraction permits one to concentrate on a problem at some level of abstraction without regard to low level detail.

  Types of abstraction: Procedural Abstraction and Data Abstraction

- **Refinement:** Process of elaboration. Refinement function defined at the abstract level, decomposes the statement of function in a stepwise fashion until programming language statements are reached.

- **Modularity:** Software is divided into separately named and addressable components called modules. Following the “divide and conquer” concept, a complex problem is broken down into several manageable pieces.

VIT®
BHOPAL
www.vitbhopal.ac.in

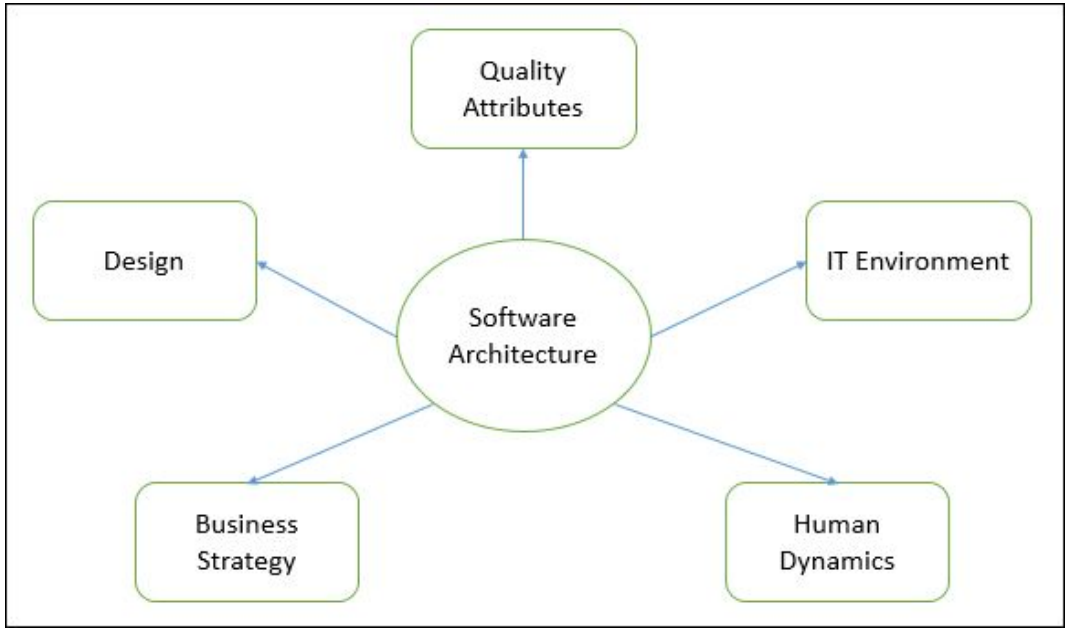
Architecture Design

Architectural Design

- Architecture design in software engineering is the process of defining the overall structure and organization of a software system.
- The architecture of a system describes its major components, their relationships (structures), and how they interact with each other.
- The primary goal of architecture design is to create a blueprint for the software system that ensures it meets its functional and non-functional requirements.
- IEEE defines architectural design as "**the process of defining a collection of hardware and software components and their interfaces to establish the framework for the development of a computer system.**"
- We can segregate Software Architecture and Design into two distinct phases: Software Architecture and Software Design. In Architecture, nonfunctional decisions are cast and in Design, functional requirements are accomplished.

Architectural Design

Software architecture and design includes several contributory factors such as Business strategy, quality attributes, human dynamics, design, and IT environment.



Architectural Design

An architectural design performs the following functions.

- Architecture serves as a blueprint for a system. It provides an abstraction to manage the system complexity and establish a communication and coordination mechanism among components.
- It defines a structured solution to meet all the technical and operational requirements, while optimizing the common quality attributes like performance and security.
- It acts as a guideline for enhancing the system (when ever required) .
- It evaluates and develops all top-level designs for the external and internal interfaces.
- It defines and documents test requirements and the schedule for software integration.

Architectural Design Representation

- **Structural model:** Illustrates architecture as an ordered collection of program components
- **Dynamic model:** Specifies the behavioural aspect of the software architecture and indicates how the structure or system configuration changes as the function changes due to change in the external environment
- **Process model:** Focuses on the design of the business or technical process, which must be implemented in the system
- **Functional model:** Represents the functional hierarchy of a system
- **Framework model:** Attempts to identify repeatable architectural design patterns encountered in similar types of application. This leads to an increase in the level of abstraction.

Architectural Design Output

- The architectural design process results in an Architectural Design Document (ADD).
- This document consists of a number of graphical representations that comprises software models along with associated descriptive text.
- The software models include static model, interface model, relationship model, and dynamic process model. They show how the system is organized into a process at run-time.
- Architectural design document gives the developers a solution to the problem stated in the Software Requirements Specification (SRS).
- Note that it considers only those requirements in detail that affect the program structure.

Architectural Design Output

In addition to ADD, other outputs of the architectural design are listed below-

- Various reports including audit report, progress report, and configuration status accounting report
- Various plans for detailed design phase, which include the following
  - Software verification and validation plan
  - Software configuration management plan
  - Software quality assurance plan
  - Software project management plan.

Architectural Styles:

The commonly used architectural styles are:

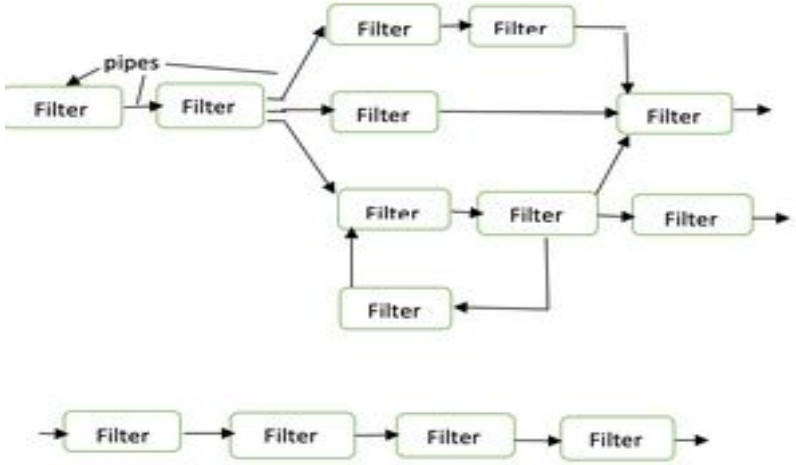
- **Data centred architectures**
- **Data flow architectures**
- **Call and Return architectures**
- **Object Oriented architecture**
- **Layered architecture:**

Architectural Styles: Data centred architectures:

- A data store will reside at the centre of this architecture and is accessed frequently by the other components that update, add, delete or modify the data present within the store.
- The figure illustrates a typical data centred style. The client software accesses a central repository.

Architectural Styles: Data flow architectures:

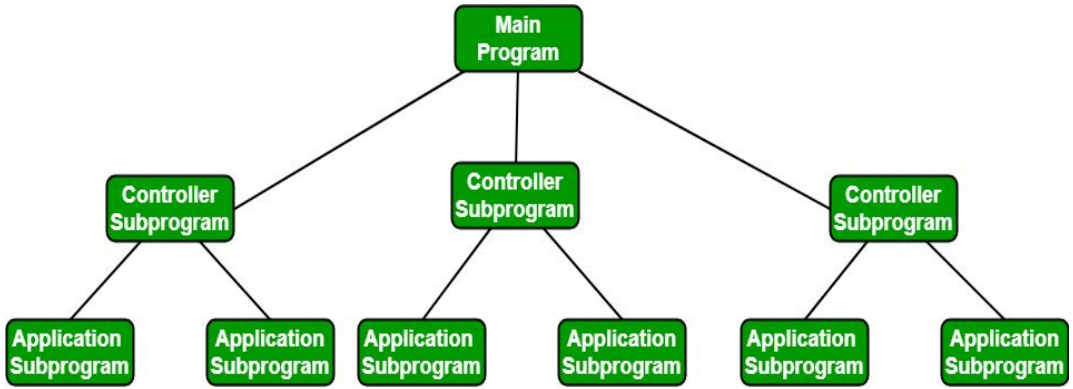
- This kind of architecture is used when input data is transformed into output data through a series of computational manipulative components.
- The figure represents pipe-and-filter architecture since it uses both pipe and filter. Pipes are used to transmit data from one component to the next.
- Each filter will work independently and is designed to take data input of a certain form and produce data output to the next filter of a specified form.



Architectural Styles: Call and Return architectures:

It is used to create a program that is easy to scale and modify. 2 Types:

- **Remote procedure call architecture:** This component is used to present in a main program or subprogram architecture distributed among multiple computers on a network.
- **Main program or Subprogram architectures:** The main program structure decomposes into a number of subprograms or functions into a control hierarchy.

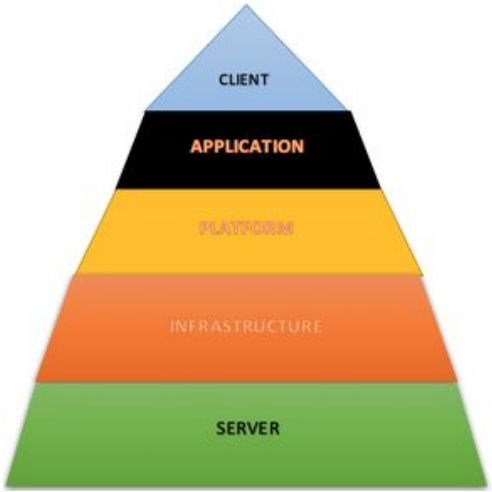


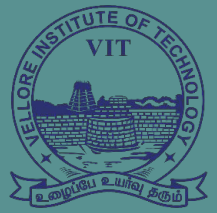
Architectural Styles: Object Oriented architecture: :

- The components of a system encapsulate data and the operations that must be applied to manipulate the data.
- The coordination and communication between the components are established via the message passing.
- It enables the designer to separate a challenge into a collection of autonomous objects.
- Other objects are aware of the implementation details of the object, allowing changes to be made without having an impact on other objects.

Architectural Styles: Layered architecture:

- A number of different layers are defined with each layer performing a well-defined set of operations.
- At the outer layer, components will receive the user interface operations and at the inner layers, components will perform the operating system interfacing
- One common example of this architectural style is the OSI-ISO system.



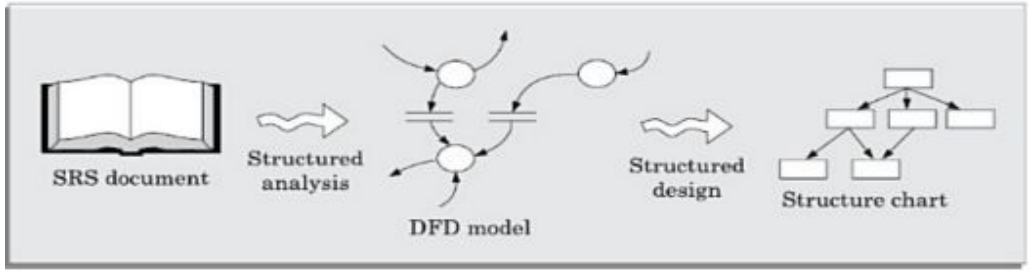


VIT®
BHOPAL
www.vitbhopal.ac.in

Structured Analysis (DFD model)
Structured Design (Structural Chart)

Structured Analysis and Structured Design (SA/SD)

- Structured Analysis and Structured Design (SA/SD) is a diagrammatic notation that is designed to help people understand the system. The basic goal of SA/SD is to improve quality and reduce the risk of system failure.
- It is based on the principle of structured programming, which emphasizes on breaking down a software system into smaller, more manageable components.
- It is divided into two phases: Structured Analysis and Structured Design.
- The roles of structured analysis (SA) and structured design (SD) have been shown schematically in below figure.



Structured Analysis and Structured Design (SA/SD)

- The structured analysis activity transforms the SRS document into a graphic model called the DFD model.
- During structured analysis, the problem is analysed, requirements are gathered, and functional decomposition of the system is achieved.
- The purpose of structured analysis is to capture the detailed structure of the system as perceived by the user.

- During structured design, all functions identified during structured analysis are mapped to a module structure, called high level design or the software architecture for the given problem.
- This is represented using a structure chart. The purpose of structured design is to define the structure of the solution that is suitable for implementation

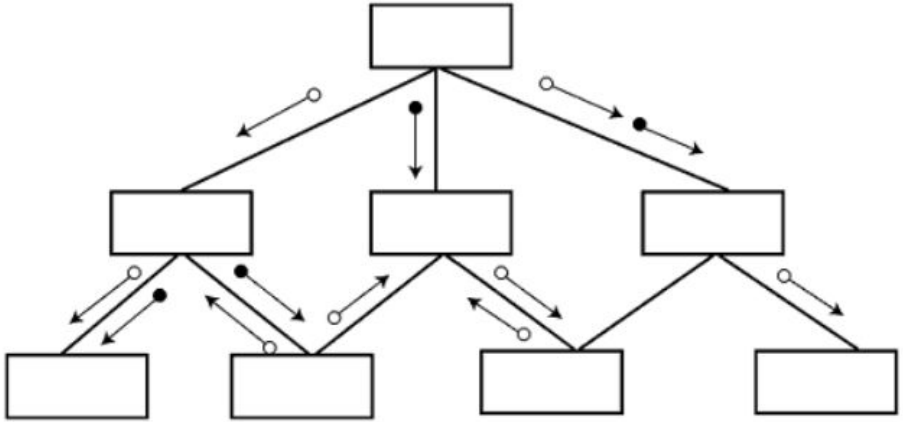
Structured Analysis and Structured Design (SA/SD)

- **Functional Decomposition:** SA/SD uses functional decomposition to break down a complex system into smaller, more manageable subsystems.
- **Data Flow Diagrams (DFDs):** SA/SD uses DFDs to model the flow of data through the system. DFDs are graphical representations of the system that show how data moves between the system's various components.
- **Data Dictionary:** A data dictionary is a central repository that contains descriptions of all the data elements used in the system.
- **Structured Design:** SA/SD uses structured design techniques to develop the system's architecture and specify the data structures and algorithms that will be used to implement the system.
- **Modular Programming:** SA/SD uses modular programming techniques to break down the system's code into smaller, more manageable modules. This makes it easier to develop, test, and maintain the system.

Structured Analysis and Structured Design (SA/SD)

SA/SD involves 2 phases:

- **Analysis Phase:** It uses
  - Data Flow Diagram,
  - Data Dictionary,
  - State Transition diagram and
  - ER diagram.
- **Design Phase:** It uses
  - Structure Chart and
  - Pseudo Code.

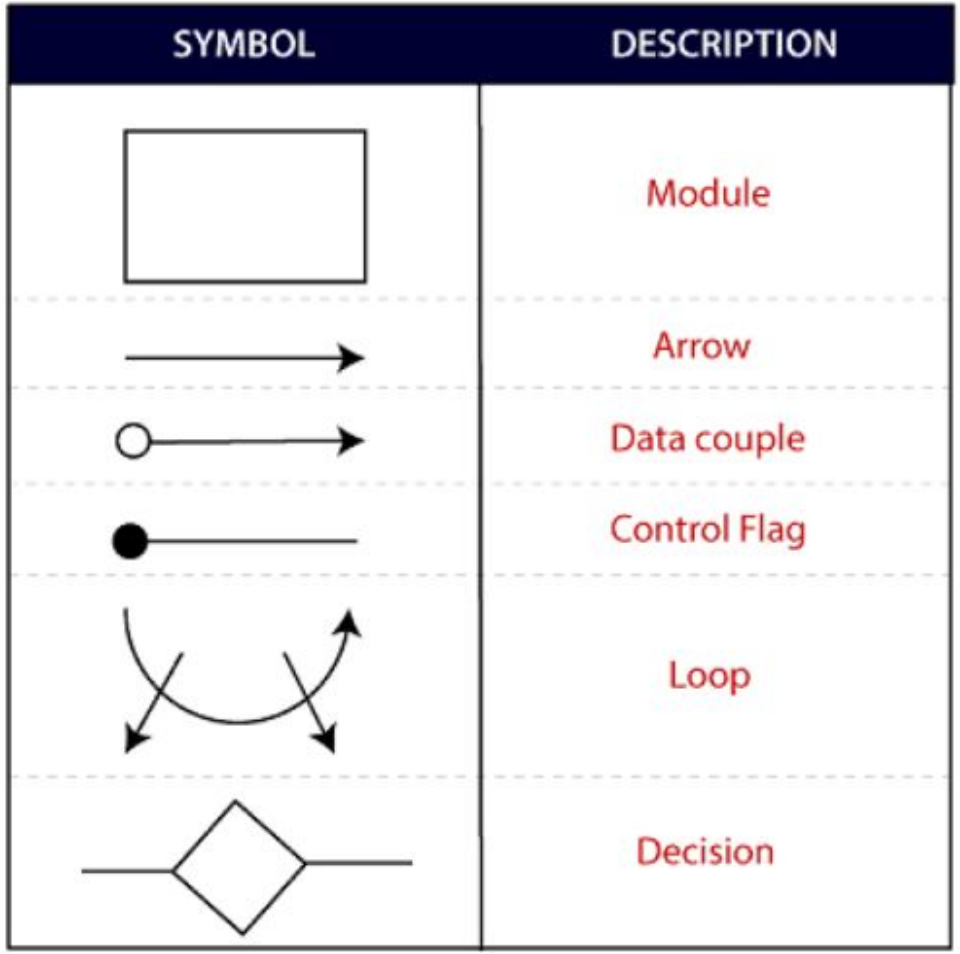


Hierarchical format of a structure chart

Structured Chart is a graphical representation which shows:

- System partitions into modules
- Hierarchy of component modules
- The relation between processing modules
- Interaction between modules
- Information passed between modules

The following notations are used in structured chart:



Structured Analysis and Structured Design (SA/SD)

Advantages of Structured Analysis and Structured Design (SA/SD):

- Clarity and Simplicity
- Better Communication
- Improved maintainability
- Better Testability

Disadvantages:

- Time-Consuming
- Inflexibility:
- Limited Iteration

VIT®
BHOPAL
www.vitbhopal.ac.in

Data Flow Diagram

Data Flow Diagrams

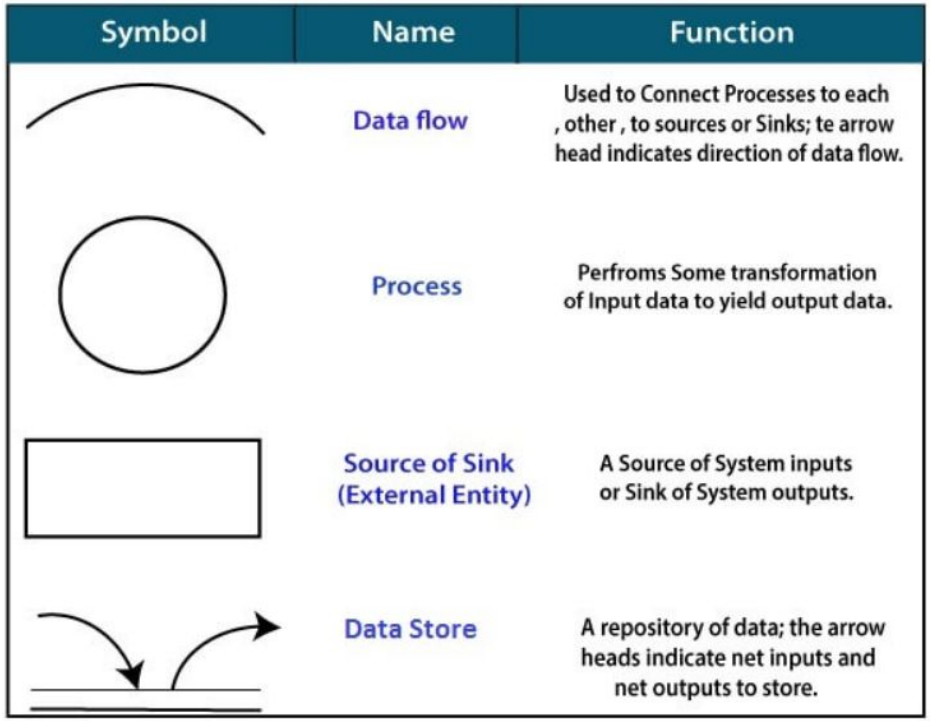
- A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It can be manual, automated, or a combination of both.
- **It shows how data enters and leaves the system, what changes the information, and where data is stored.**
- The objective of a DFD is to show the scope and boundaries of a system as a whole.
- It may be used as a communication tool between a system analyst and any person who plays a part in the order that acts as a starting point for redesigning a system.

Data Flow: A curved line shows the flow of data into or out of a process or data store.

Circle: A circle (bubble) shows a process that transforms data inputs into data outputs.

Source or Sink: Source or Sink is an external entity and acts as a source of system inputs or sink of system outputs.

Data Store: A set of parallel lines shows a place for the collection of data items. A data store indicates that the data is stored which can be used at a later stage or by the other processes in a different order. The data store can have an element or group of elements.



Symbols for Data Flow Diagrams

Levels in Data Flow Diagrams (DFD)

- DFDs may be partitioned into levels that represent increasing information flow and functional detail.
- Levels in DFD are numbered 0, 1, 2 or beyond. Here, we will see primarily three levels in the data flow diagram, which are:
  - 0-level DFD,
  - 1-level DFD, and
  - 2-level DFD.

0-level DFDM

- It is also known as fundamental system model, or context diagram
- It represents the entire software requirement as a single bubble with input and output data denoted by incoming and outgoing arrows.
- Then the system is decomposed and described as a DFD with multiple bubbles. Thus, if bubble "A" has two inputs x1 and x2 and one output y, then the expanded DFD, that represents "A" should have exactly two external inputs and one external output as shown in fig:

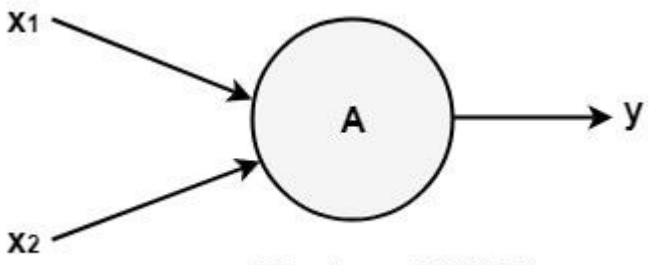


Fig: Level-0 DFD.

0-level DFDM

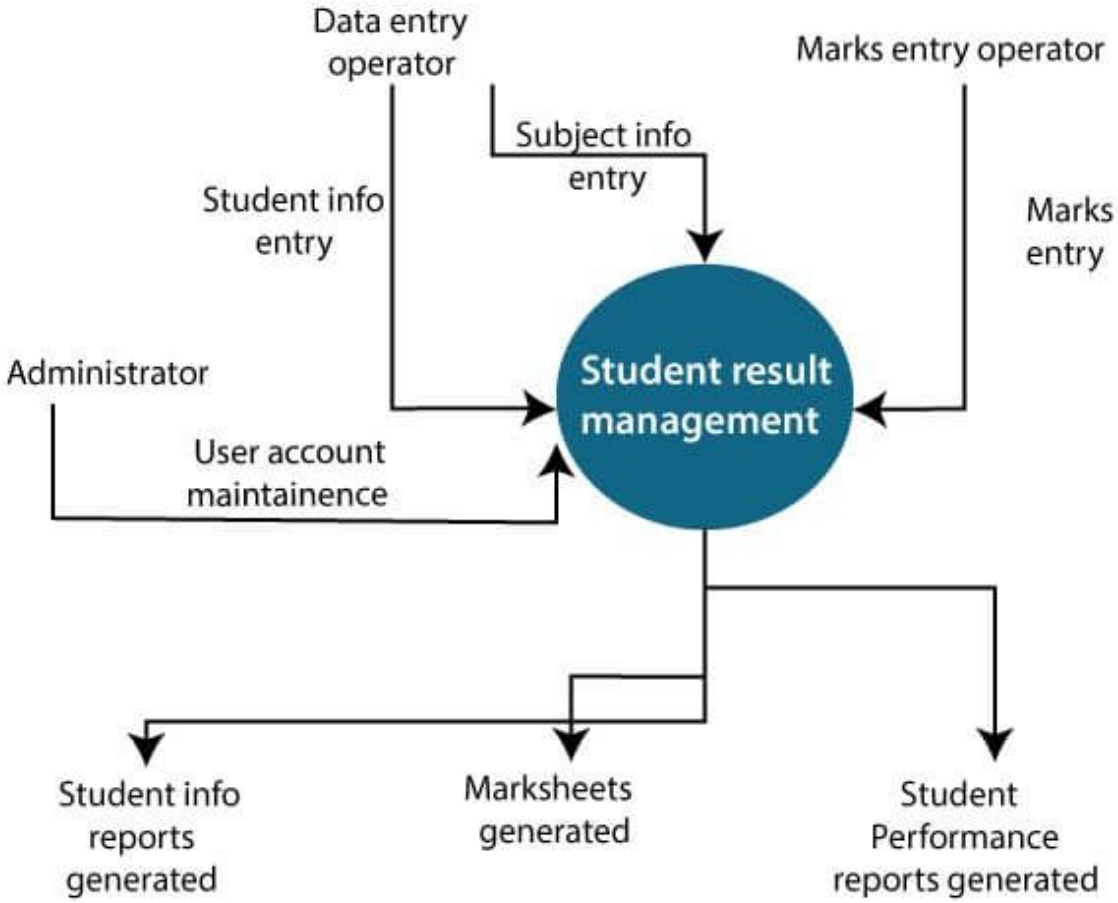


Fig: Level-0 DFD of result management system

1-level DFDM

- In 1-level DFD, a context diagram is decomposed into multiple bubbles/processes.
- In this level, we highlight the main objectives of the system and breakdown the high-level process of 0-level DFD into sub-processes.

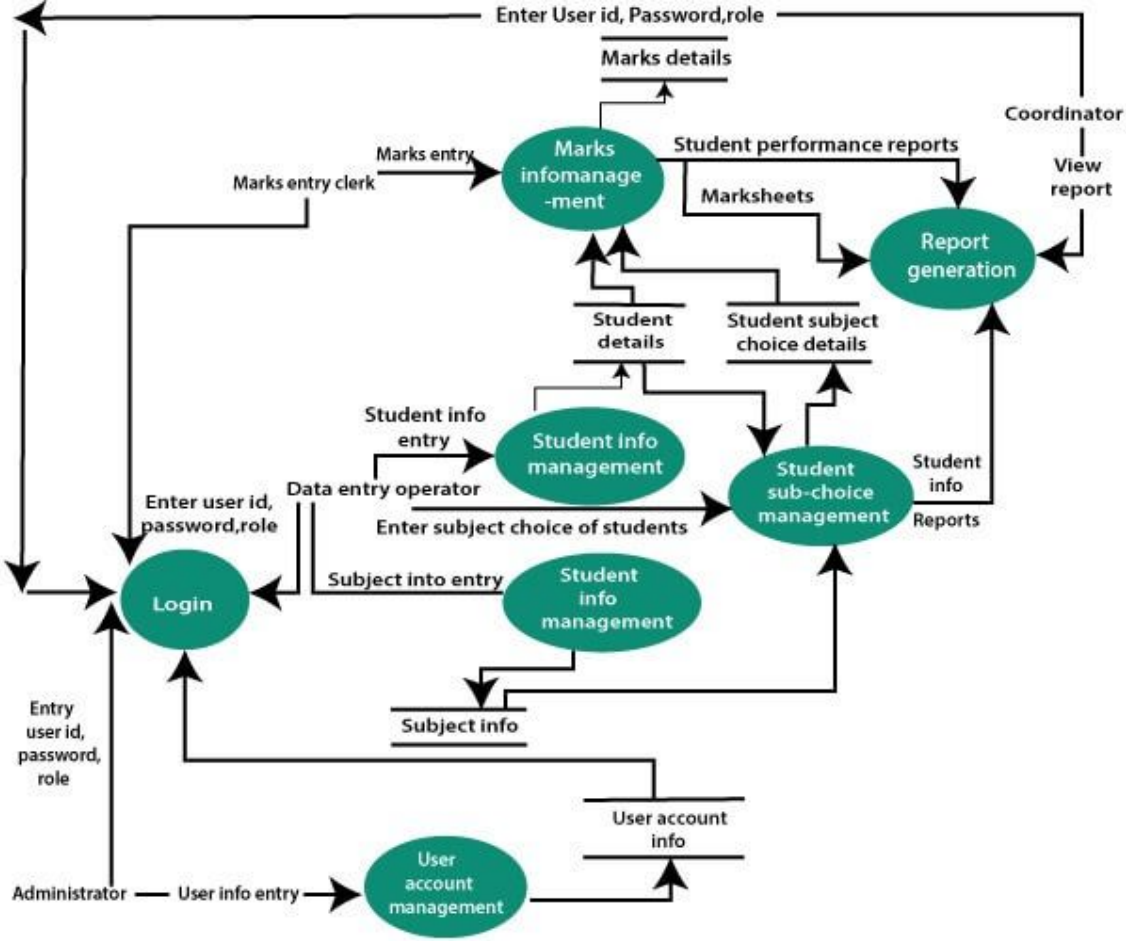
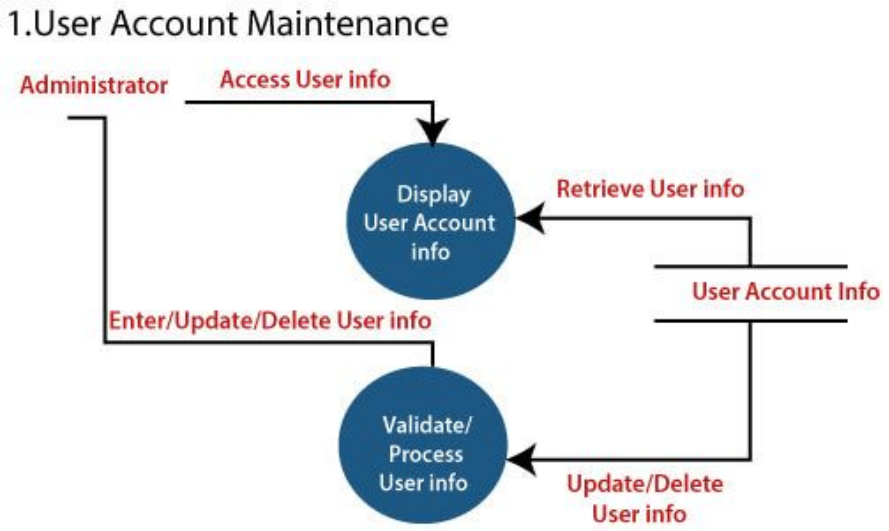
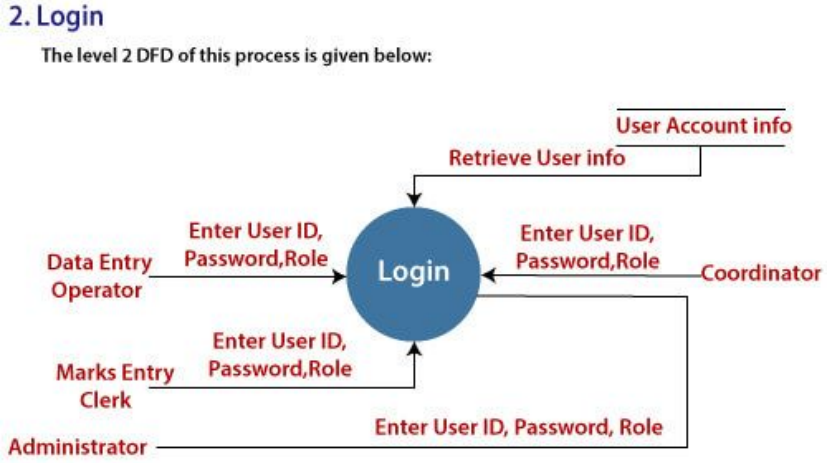


Fig: Level-1 DFD of result management system

2-level DFDM

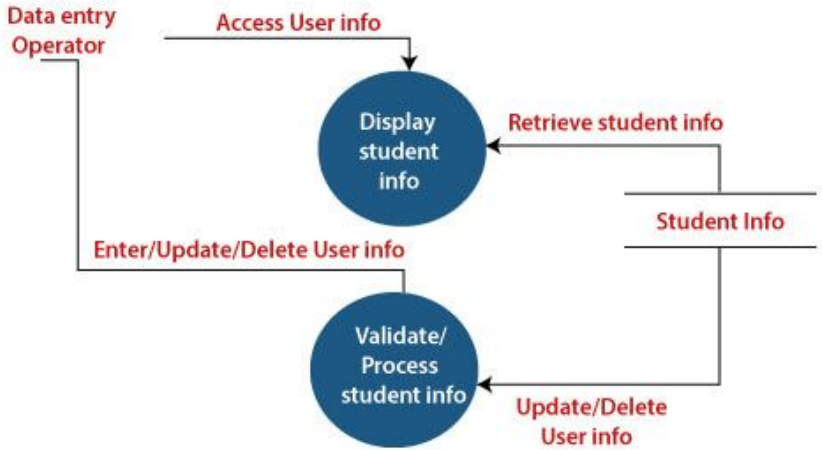
- 2-level DFD goes one process deeper into parts of 1-level DFD. It can be used to project or record the specific/necessary detail about the system's functioning.





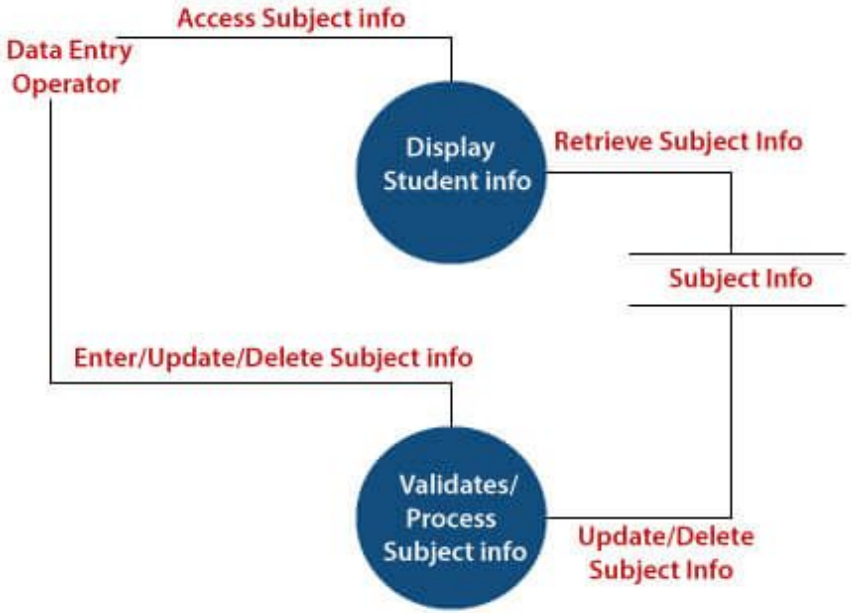
2-level DFDM

3. Student Information Management



4. Subject Information Management

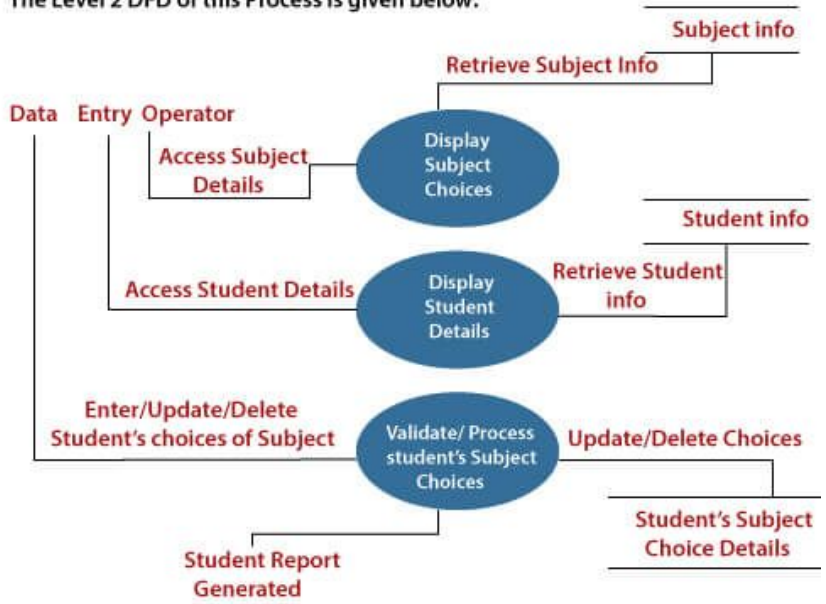
The level 2 DFD of this process is given below:



2-level DFDM

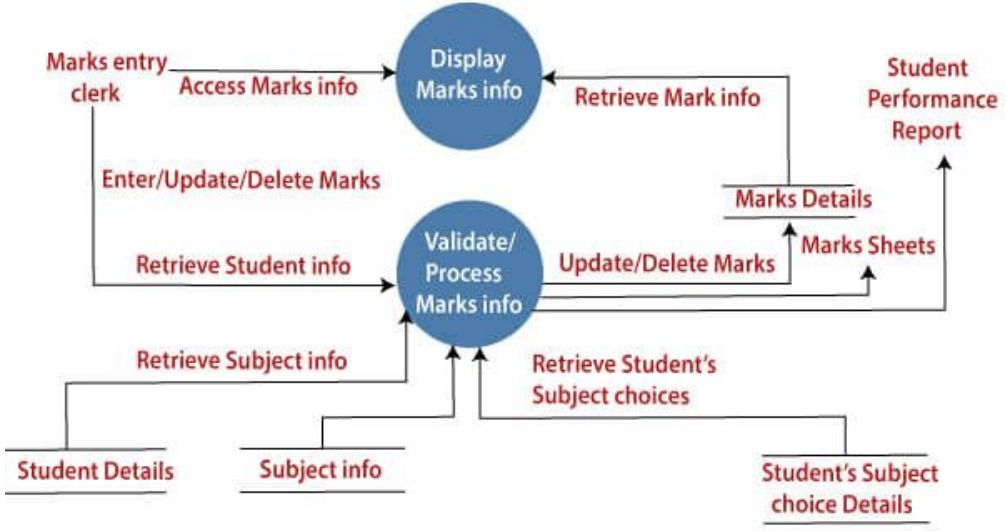
5. Student's Subject Choice Management

The Level 2 DFD of this Process is given below:



6. Marks Information Managment

The Level 2 DFD of this Process is given below:



VIT®
BHOPAL
www.vitbhopal.ac.in

Structural Chart

Structural Chart

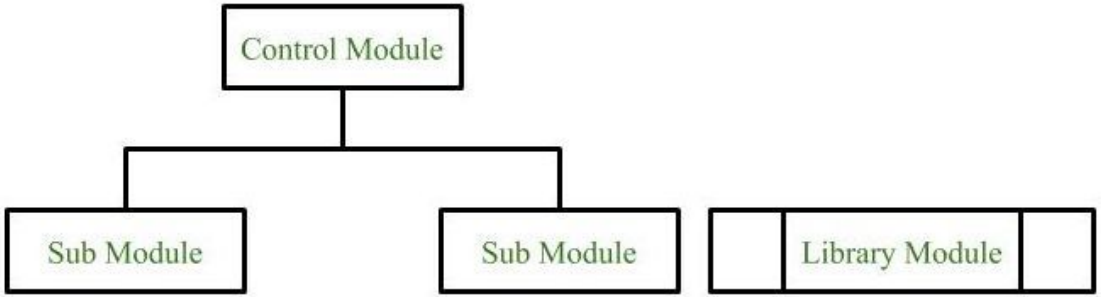
- Structure Chart represent hierarchical structure of modules. It breaks down the entire system into lowest functional modules, describe functions and sub-functions of each module of a system to a greater detail.
- Structure Chart partitions the system into black boxes (functionality of the system is known to the users but inner details are unknown). Inputs are given to the black boxes and appropriate outputs are generated.
- Components are read from top to bottom and left to right. When a module calls another, it views the called module as black box, passing required parameters and receiving results.

Structural Chart

Symbols used in construction of structured chart

1. Module: It represents the process or task of the system. It is of three types.

- Control Module: A control module branches to more than one sub module.
- Sub Module: Sub Module is a module which is the part (Child) of another module.
- Library Module: Library Module are reusable and invokable from any module.

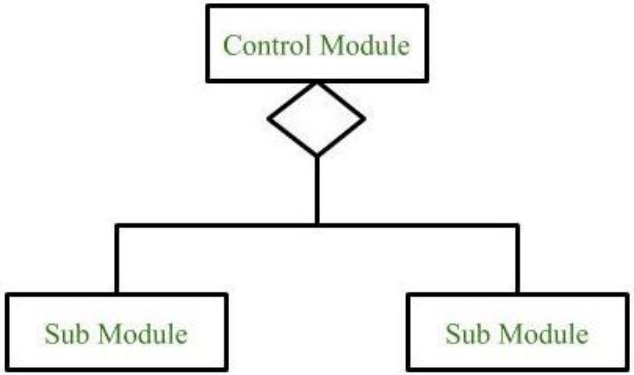


Structural Chart

Symbols used in construction of structured chart

2. Conditional Call

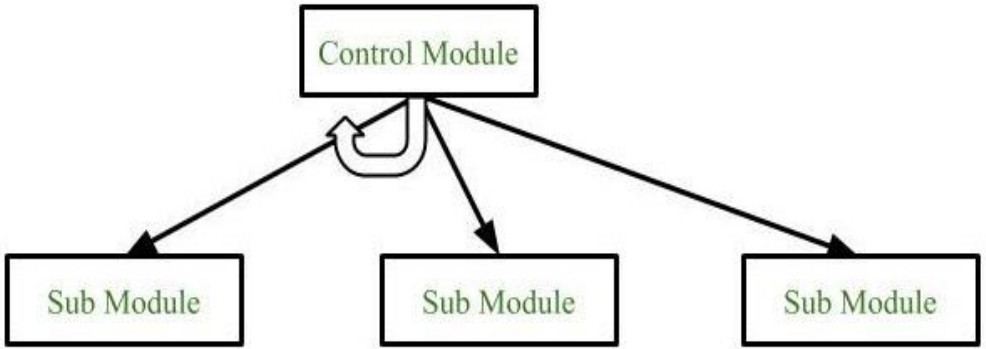
It represents that control module can select any of the sub module on the basis of some condition.



3. Loop (Repetitive call of module)

It represents the repetitive execution of module by the sub module.

A curved arrow represents loop in the module.

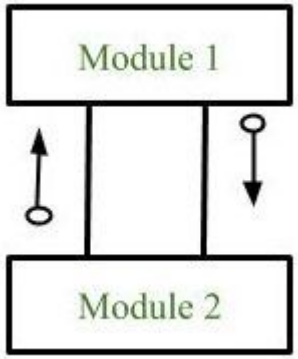


Structural Chart

Symbols used in construction of structured chart

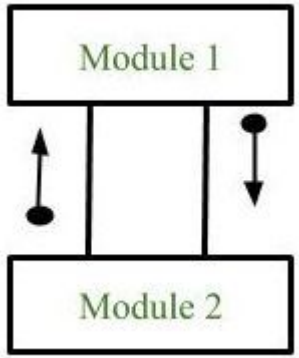
4. Data Flow

It represents the flow of data between the modules. It is represented by directed arrow with empty circle at the end.



5. Control Flow

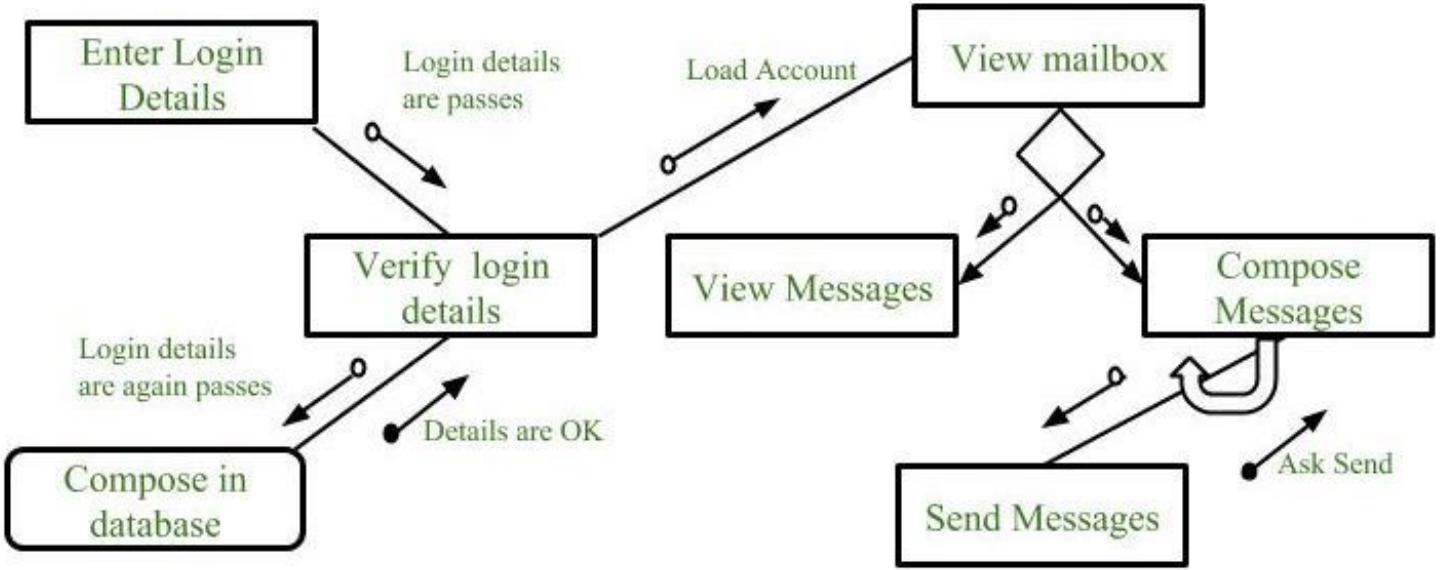
It represents the flow of control between the modules. It is represented by directed arrow with filled circle at the end.



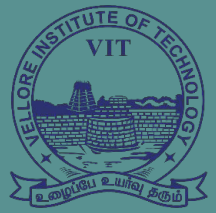
6. Physical Storage

Physical Storage is that where all the information are to be stored.

Structural Chart



Example : Structure chart for an Email server



VIT®
BHOPAL
www.vitbhopal.ac.in

User Interface Design

User Interface Design

- User interface is the front-end application view to which user interacts in order to use the software.
- Users can manipulate and control the software as well as hardware by means of user interface.
- Today, user interfaces are found at almost every place where digital technology exists, right from computers, mobile phones, cars, music players, airplanes, ships etc.
- User interface is part of software and is designed such a way that it is expected to provide the user insight of the software and it provides a fundamental platform for human-computer interaction.
- User interfaces can be graphical, text-based, audio-video based, depending upon the underlying hardware and software combination.

User Interface Design

The software becomes more popular if its user interface is:

- Attractive
- Simple to use
- Responsive in short time
- Clear to understand
- Consistent on all interfacing screens

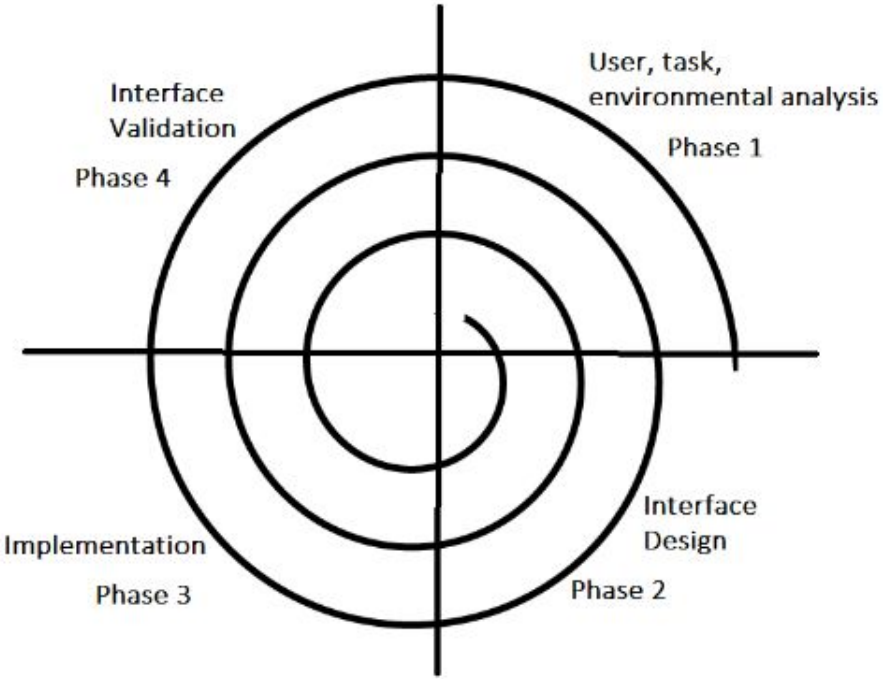
User interface is broadly divided into two categories:

- **Command Line Interface:** Command Line Interface provides a command prompt, where the user types the command and feeds to the system. The user needs to remember the syntax of the command and its use.

- **Graphical User Interface:** Graphical User Interface provides the simple interactive interface to interact with the system. GUI can be a combination of both hardware and software. Using GUI, user interprets the software.

User Interface Design Process

The analysis and design process of a user interface is iterative and can be represented by a spiral model.



It consists of four framework activities.

1. User, task, environmental analysis, and modelling
2. Interface Design
3. Interface construction and implementation
4. Interface Validation

User Interface Design Process

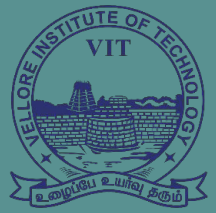
1. **User, task, environmental analysis, and modelling:** Initially, the focus is based on the profile of users who will interact with the system, i.e. understanding, skill and knowledge, type of user, etc. The analysis of the user environment focuses on the physical work environment.
2. **Interface Design:** The goal of this phase is to define the set of interface objects and actions i.e. Control mechanisms that enable the user to perform desired tasks.
3. **Interface construction and implementation:** The implementation activity begins with the creation of prototype (model) that enables usage scenarios to be evaluated.
4. **Interface Validation:** This phase focuses on testing the interface. The interface should be perform tasks correctly and should achieve all the user's requirements. It should be easy to use and easy to learn.

User Interface Design Principles

User interface design is a key component of software engineering, as it can have a significant impact on the usability, effectiveness, and user experience of an application.

There are several key principles that software engineers should follow when designing user interfaces:

- User-centred design
- Consistency
- Simplicity
- Feedback
- Accessibility
- Flexibility



VIT®
BHOPAL
www.vitbhopal.ac.in

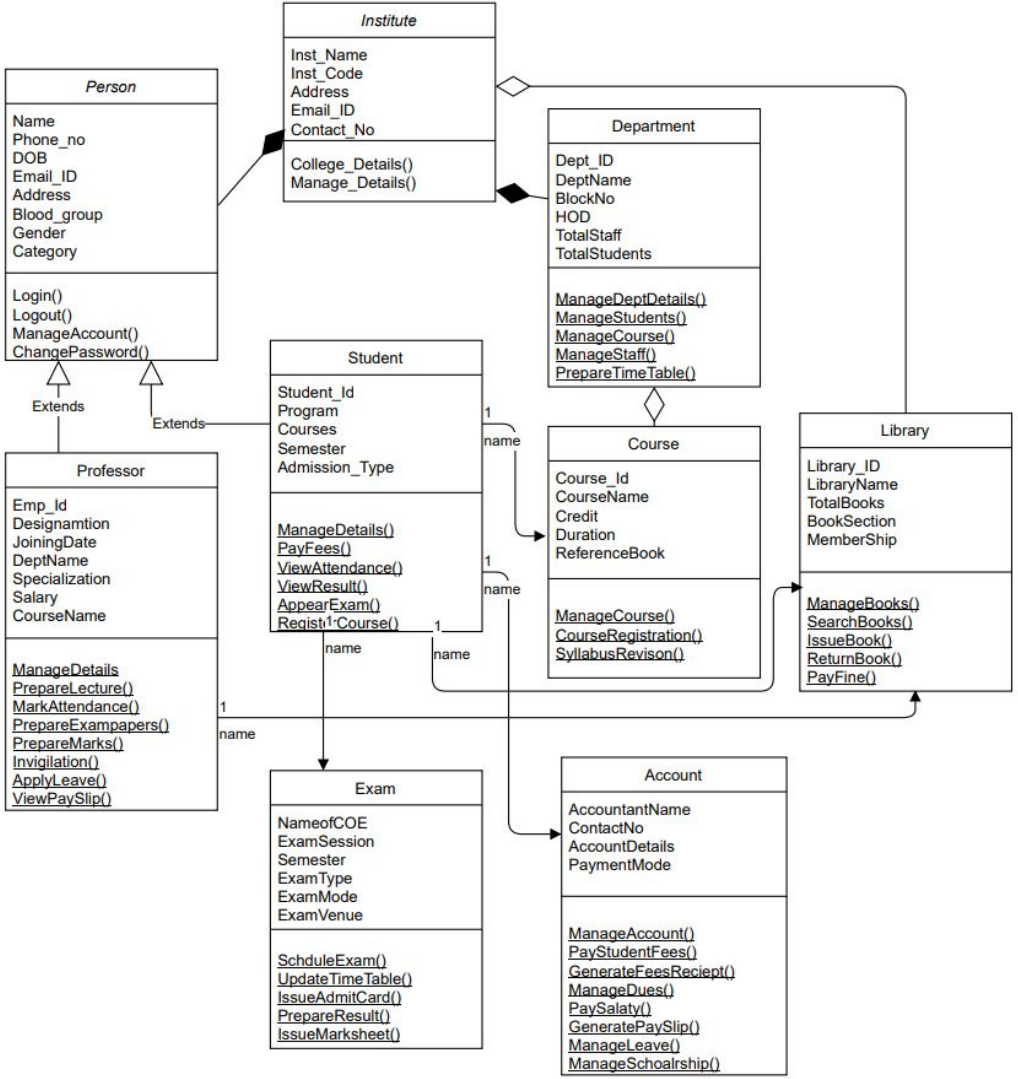
UML Diagrams

Use Case Diagram

Institute Management System

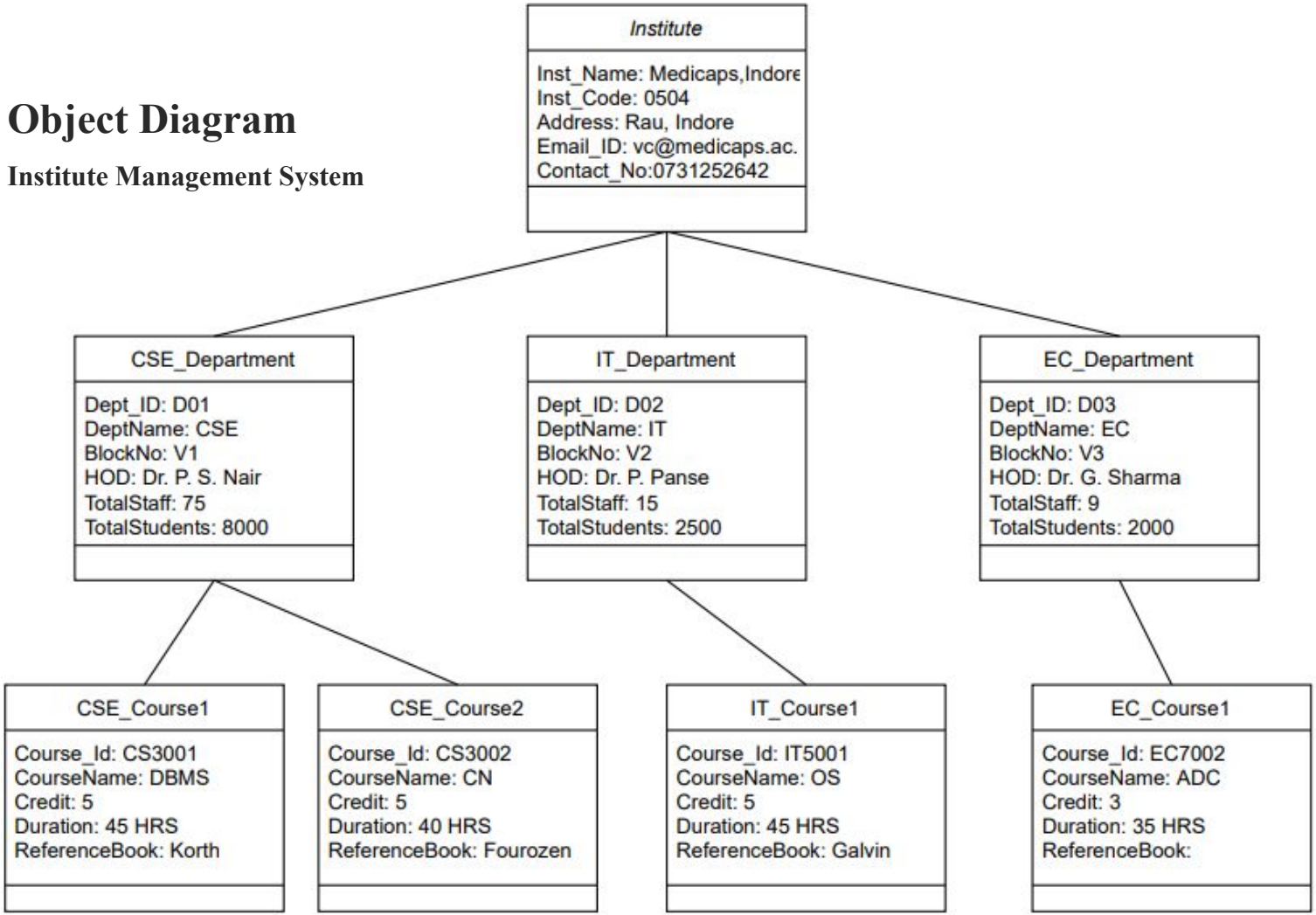
Class Diagram

Institute Management System



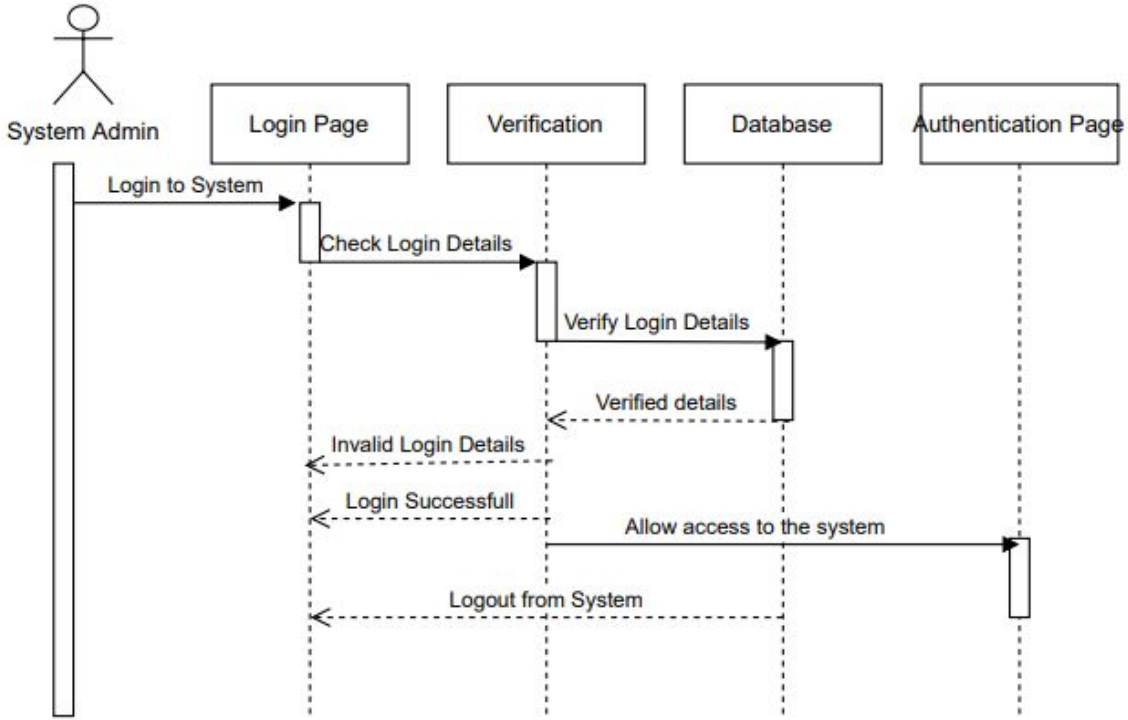
Object Diagram

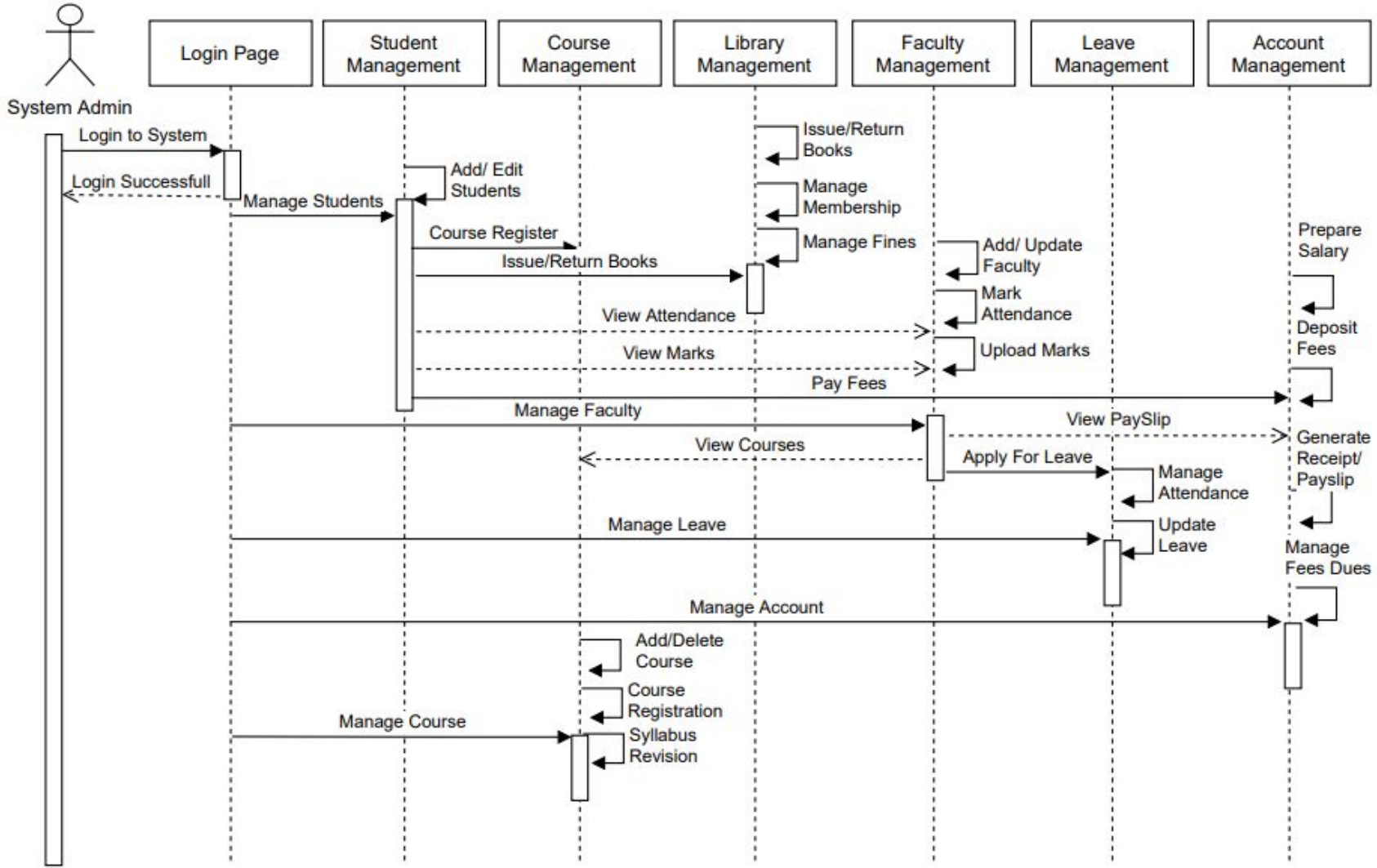
Institute Management System



Sequence Diagram

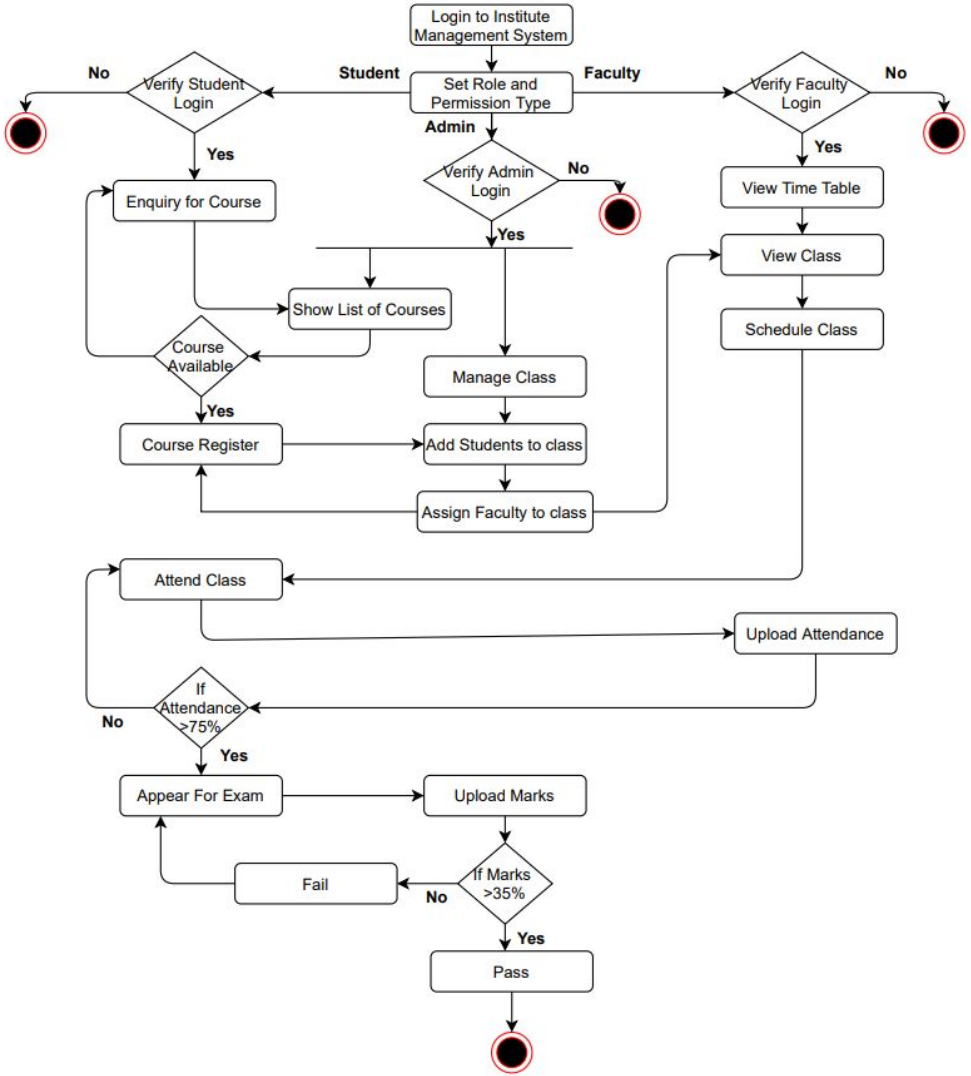
Institute Management System




System Admin
Login Page
Student Management
Course Management
Library Management
Faculty Management
Leave Management
Account Management
Login to System
Login Successfull
Manage Students
Add/ Edit Students
Course Register
Issue/Return Books
Issue/Return Books
Manage Membership
Manage Fines
Add/ Update Faculty
Mark Attendance
Upload Marks
View Attendance
View Marks
Pay Fees
Prepare Salary
Deposit Fees
Manage Faculty
View PaySlip
View Courses
Apply For Leave
Manage Attendance
Generate Receipt/ Payslip
Manage Leave
Update Leave
Manage Fees Dues
Manage Account
Add/Delete Course
Course Registration
Syllabus Revision
Manage Course

Activity Diagram

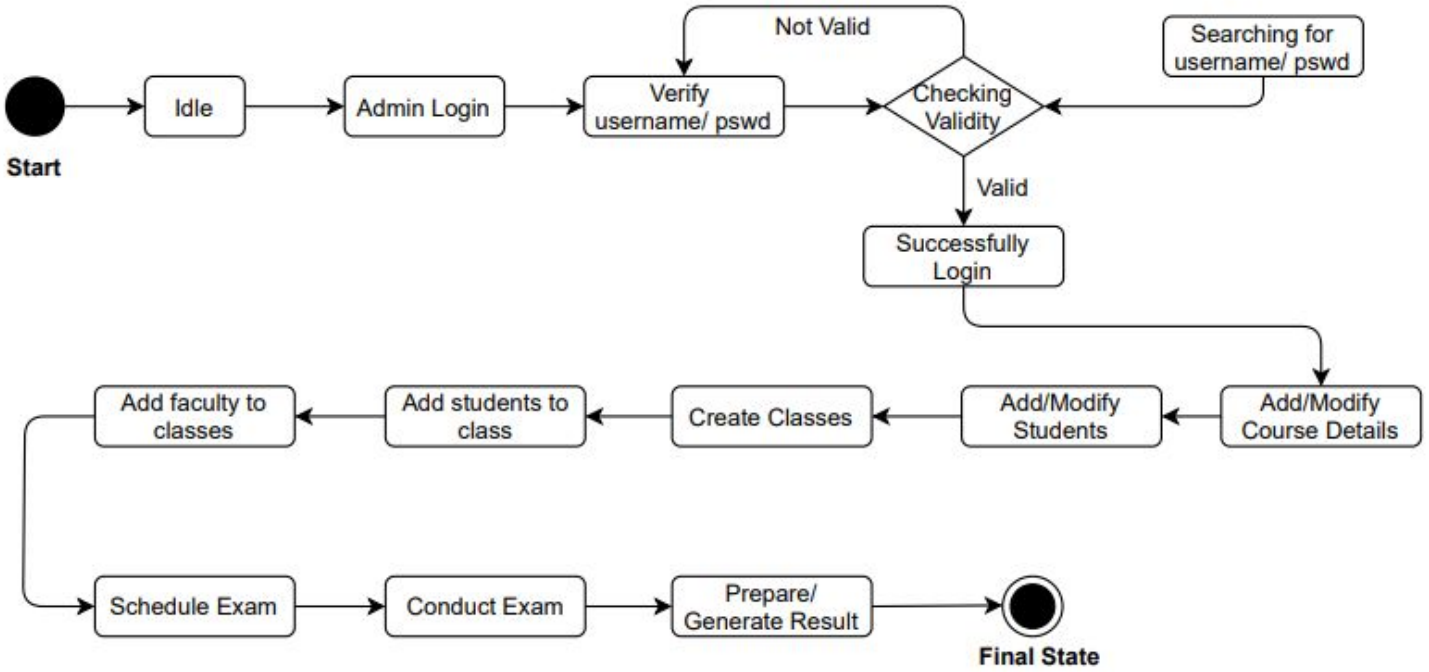
Institute Management System



State Chart Diagram

Institute Management System

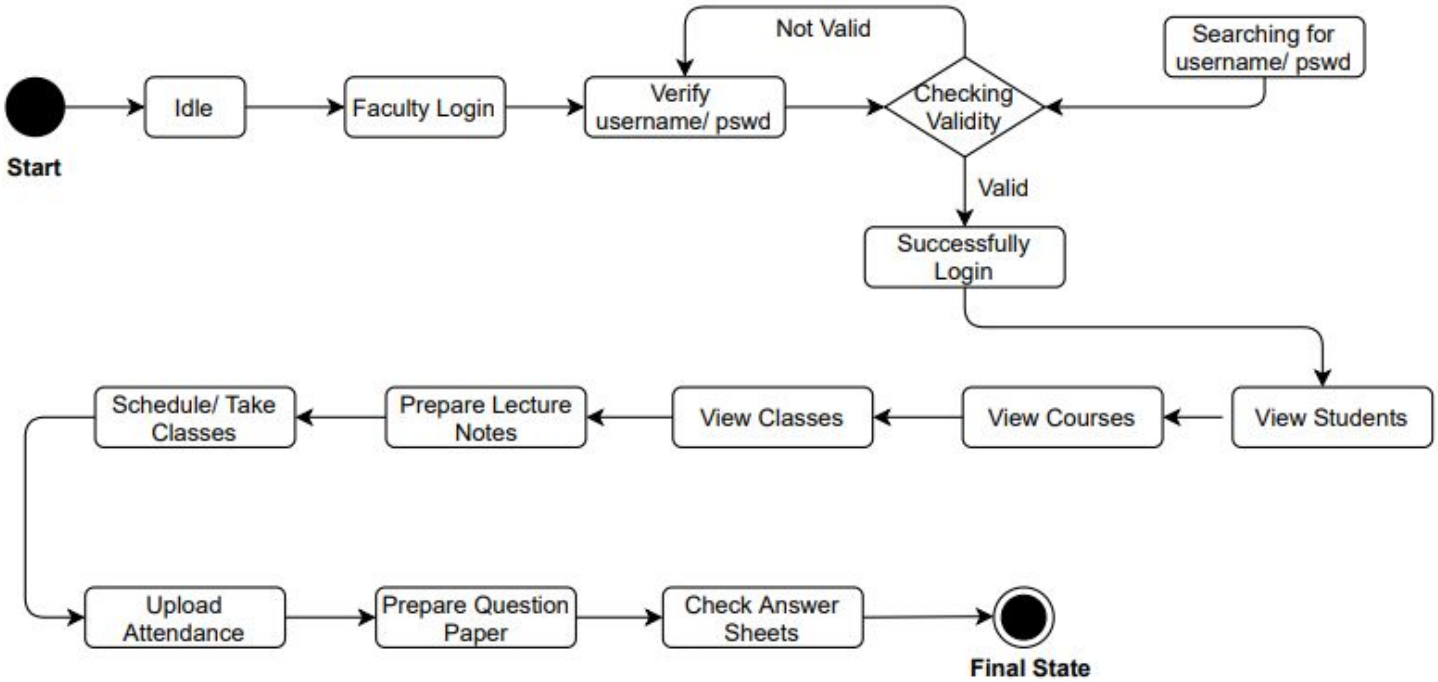
For Admin



State Chart Diagram

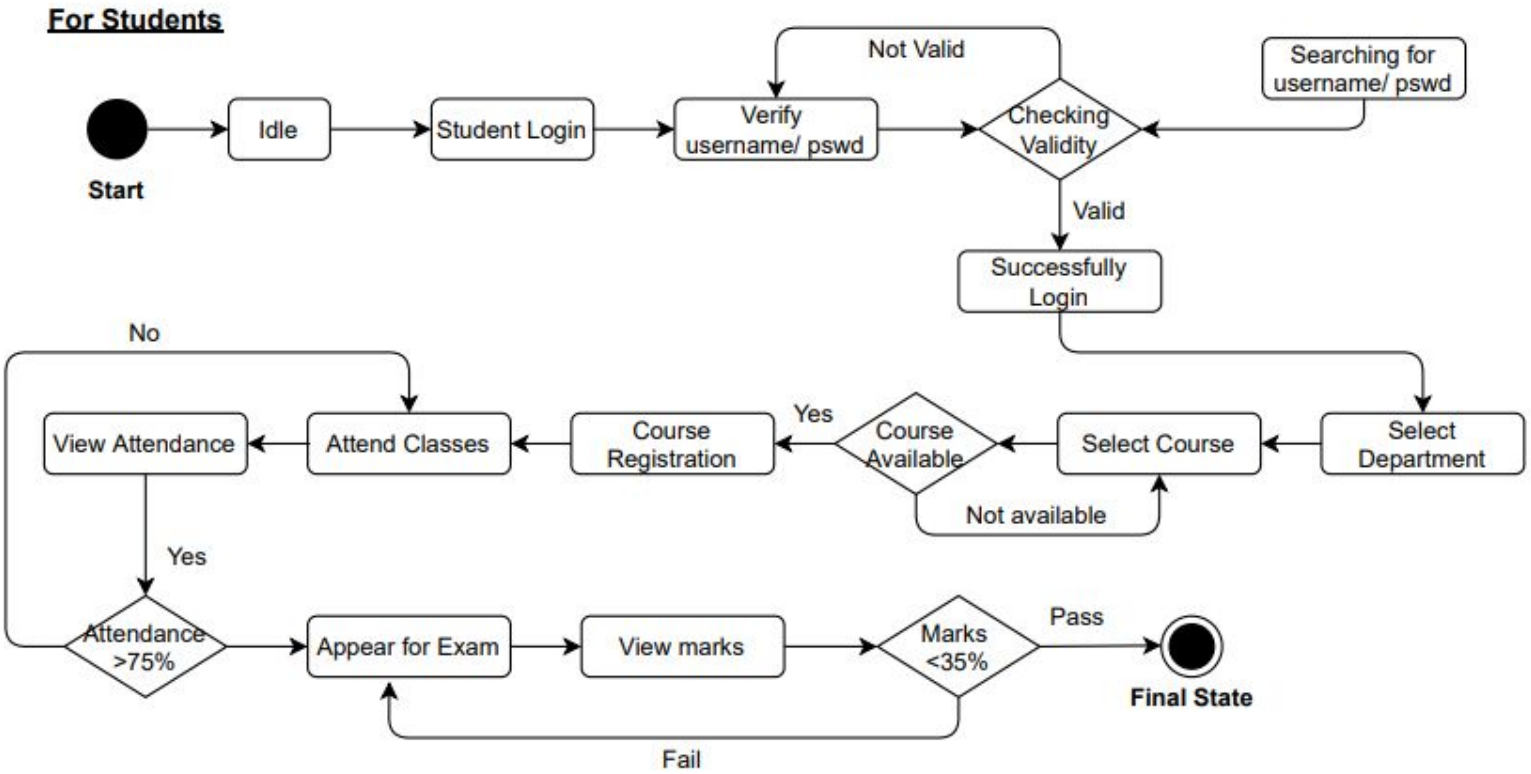
Institute Management System

For Faculty



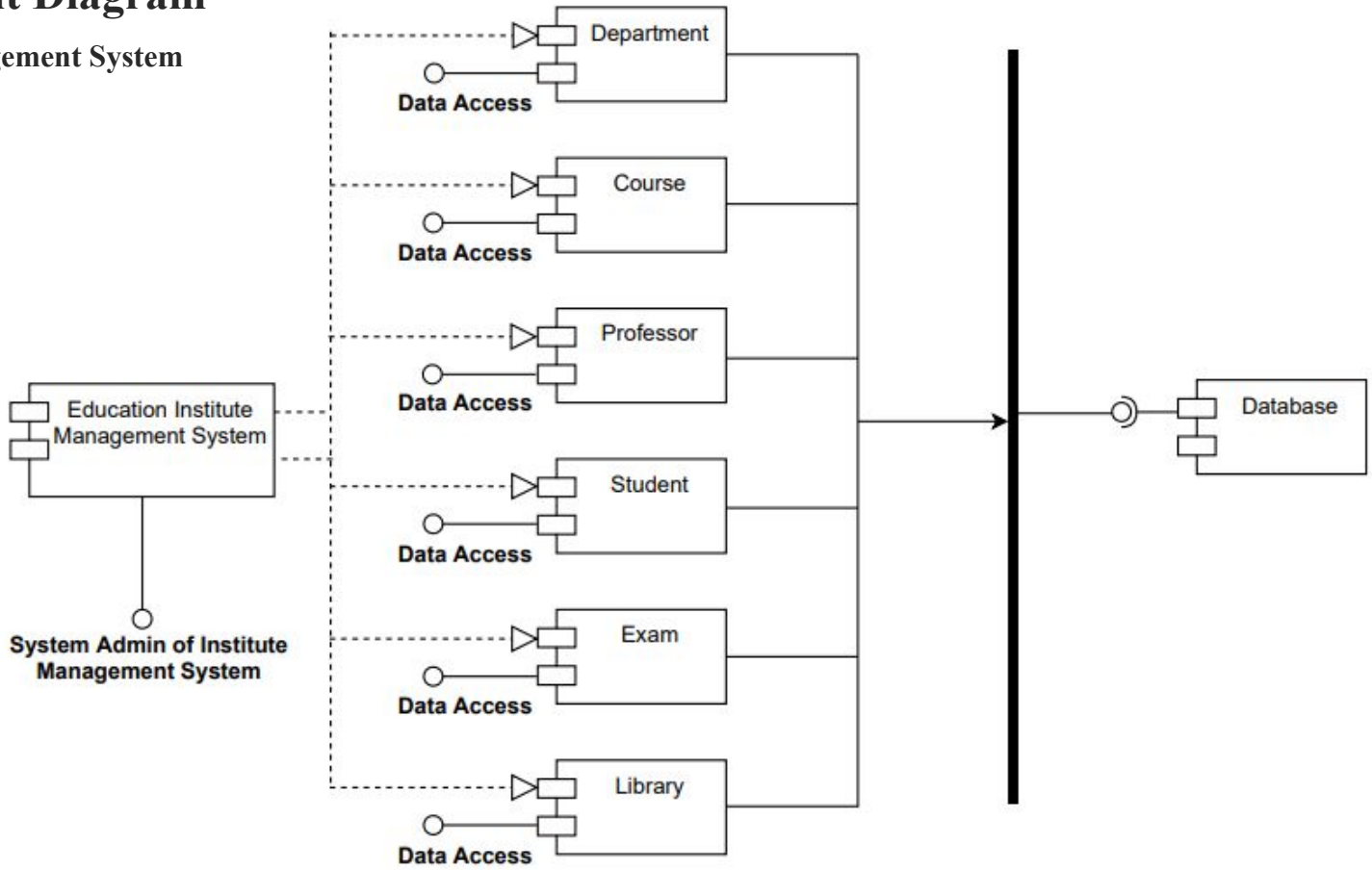
State Chart Diagram

Institute Management System



Component Diagram

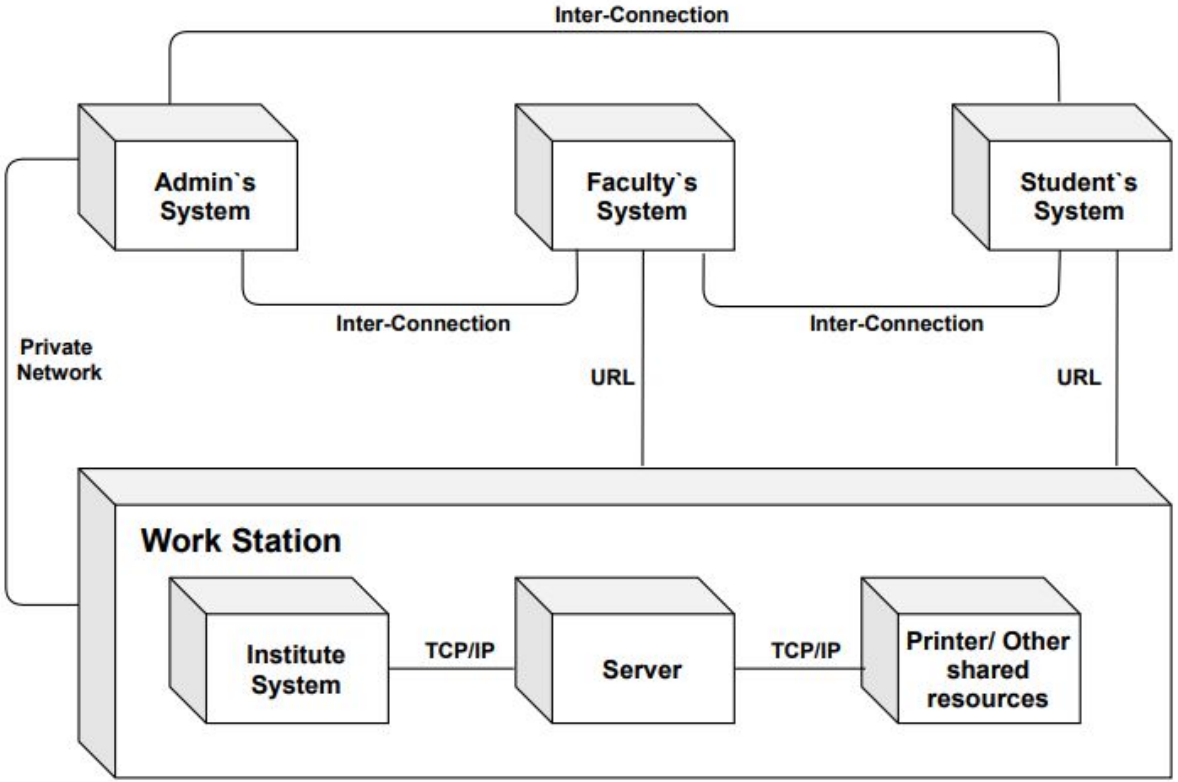
Institute Management System



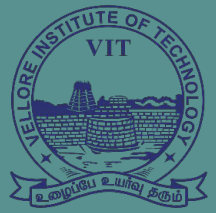
Component Diagram: Educational Institute Management System

Deployment Diagram

Institute Management System



Deployment Diagram: Educational Institute Management System



VIT®
BHOPAL
www.vitbhopal.ac.in

Software Testing

Software Testing Fundamentals

- **Software Testing is the process of exercising or evaluating a system or system components by manual or automated means to verify that it satisfies specified requirements.**
- Software testing can be stated as the process of verifying and validating whether a software or application is
  - bug-free,
  - meets the technical requirements as guided by its design and development, and
  - meets the user requirements effectively and efficiently by handling all the exceptional and boundary cases.
- The process of software testing aims not only at finding faults in the existing software but also at finding measures to improve the software in terms of efficiency, accuracy, and usability.

Why is Software Testing?

- Software Testing is a method to assess the functionality of the software program. The process verifies whether the actual software meets the expected requirements and ensures the software is free from bugs.
- The purpose of software testing is to identify the errors, faults, or missing requirements in contrast to actual requirements. It primarily aims to measure the specifications, functionality, and performance of a software program or application.

Software testing can be divided into two steps:

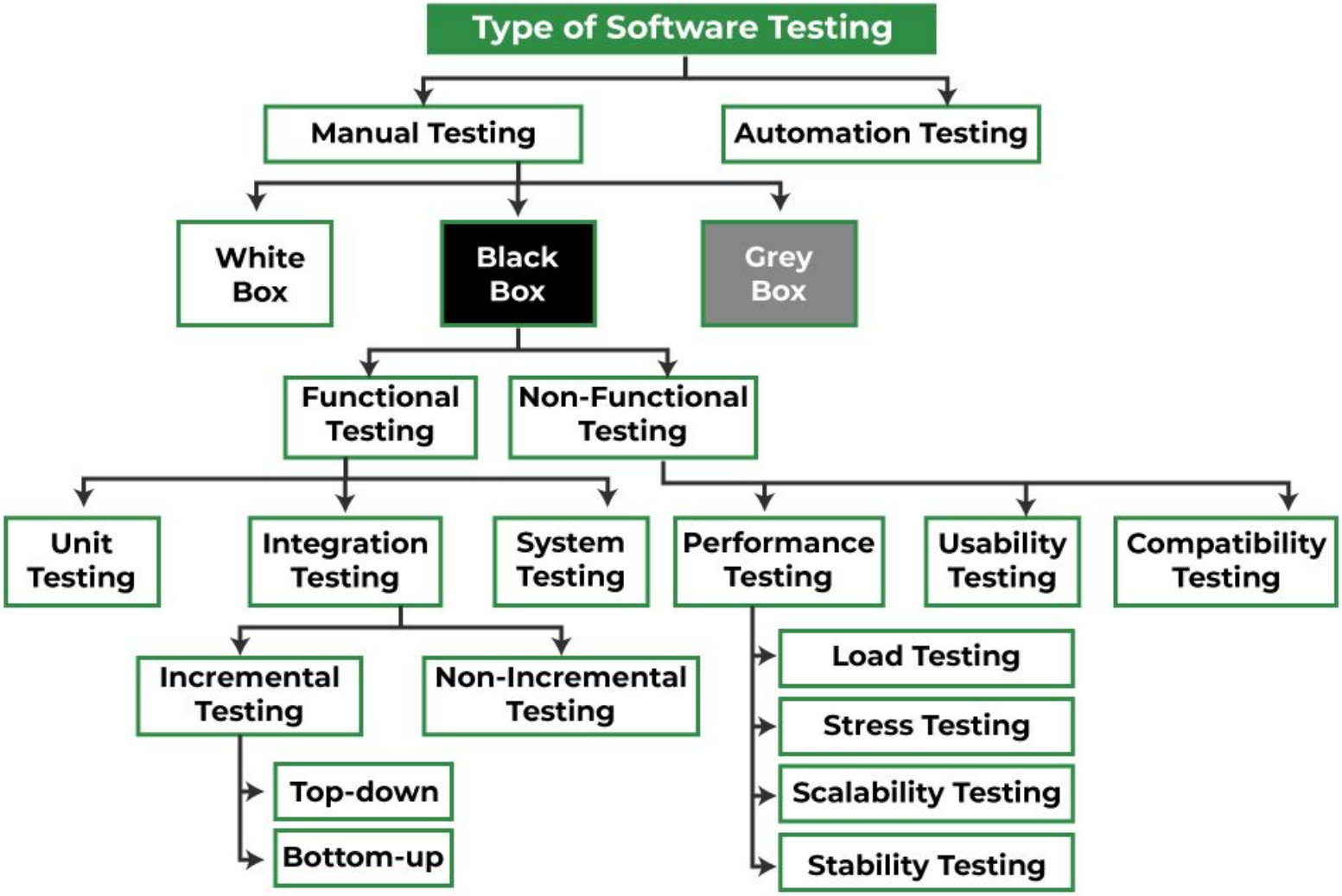
- **Verification:** It refers to the set of tasks that ensure that the software correctly implements a specific function. It means "***Are we building the product right?".***
- **Validation:** It refers to a different set of tasks that ensure that the software that has been built is traceable to customer requirements. It means "***Are we building the right product?"***

Importance of Software Testing:

- **Defects can be identified early:** **B**ugs can be identified early and can be fixed before the delivery of the software.
- **Improves quality of software:** Software Testing uncovers the defects in the software, and fixing them improves the quality of the software.
- **Increased customer satisfaction:** Software testing ensures reliability, security, and high performance which results in saving time, costs, and customer satisfaction.
- **Helps with scalability:** Software testing type non-functional testing helps to identify the scalability issues and the point where an application might stop working.
- **Saves time and money:** After the application is launched it will be very difficult to trace and resolve the issues, as performing this activity will incur more costs and time. Thus, it is better to conduct software testing at regular intervals during software development.

System Testing Examples and Use Cases

- **Software Applications:** Use cases for an online airline's booking system – customers browse flight schedules and prices, select dates and times, etc.
- **Web Applications:** An e-commerce company lets you search and filter items, select an item, add it to the cart, purchase it, and more.
- **Mobile Applications:** An UPI app let you do mobile recharge or transfer money securely. So, first, you have to select the mobile number, then the biller name, recharge amount, and payment method, and proceed to pay.
- **Games:** For a gaming app, check the animation, landscape-portrait orientation, background music, sound on/off, score, leaderboard, etc.
- **Operating Systems:** Login to the system with your password, check your files, folders, apps are well placed and working, battery percentage, time-zone, go to the 'settings' for additional checkups, etc.
- **Hardware:** Test the mechanical parts – speed, temperature, etc., electronic parts – voltage, currents, power input-output, communication parts- bandwidths, etc.


Type of Software Testing
Manual Testing
Automation Testing
White Box
Black Box
Grey Box
Functional Testing
Non-Functional Testing
Unit Testing
Integration Testing
System Testing
Performance Testing
Usability Testing
Compatibility Testing
Incremental Testing
Non-Incremental Testing
Top-down
Bottom-up
Load Testing
Stress Testing
Scalability Testing
Stability Testing

Different Types Of Software Testing

- **Manual Testing:**
  - Manual testing includes testing software manually, i.e., without using any automation tool or script.
  - In this type, the tester takes over the role of an end-user and tests the software to identify any unexpected behaviour or bug.
  - There are different stages for manual testing such as unit testing, integration testing, system testing, and user acceptance testing. Testers use test plans, test cases, or test scenarios to test software to ensure the completeness of testing.
- **Automation Testing:**
  - Automation testing, which is also known as Test Automation, is when the tester writes scripts and uses another software to test the product.
  - This process involves the automation of a manual process. Automation Testing is used to re-run the test scenarios quickly and repeatedly, that were performed manually in manual testing.

Different Types Of Software Testing

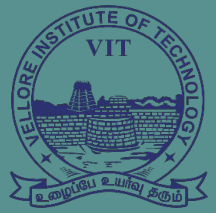
Software testing techniques can be majorly classified into two categories:

- **Black Box Testing:** Black box technique of testing in which the tester doesn't have access to the source code of the software and is conducted at the software interface without any concern with the internal logical structure of the software known as black-box testing.
- **White-Box Testing:** White box technique of testing in which the tester is aware of the internal workings of the product, has access to its source code, and is conducted by making sure that all internal operations are performed according to the specifications is known as white box testing.
- **Grey Box Testing:** Grey Box technique is testing in which the testers should have knowledge of implementation, however, they need not be experts.

Different Types Of Software Testing

Software Testing can be broadly classified into 3 types:

- **Functional Testing:** Functional testing is a type of software testing that validates the software systems against the functional requirements. It is performed to check whether the application is working as per the software's functional requirements or not. Various types of functional testing are Unit testing, Integration testing, System testing, Smoke testing, and so on.
- **Non-functional Testing:** Non-functional testing is a type of software testing that checks the application for non-functional requirements like performance, scalability, portability, stress, etc. Various types of non-functional testing are Performance testing, Stress testing, Usability Testing, and so on.
- **Maintenance Testing:** Maintenance testing is the process of changing, modifying, and updating the software to keep up with the customer's needs. It involves regression testing that verifies that recent changes to the code have not adversely affected other previously working parts of the software.



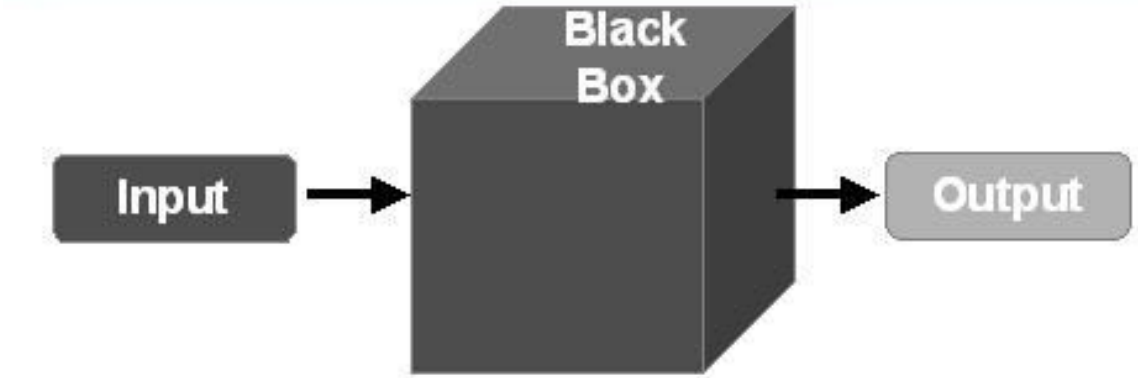
VIT®
BHOPAL
www.vitbhopal.ac.in

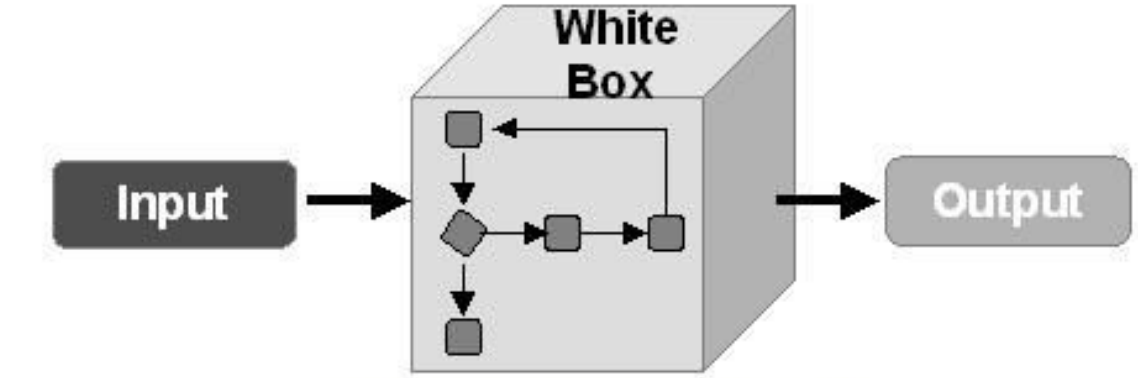
Black Box and White Box Testing

Different Types Of Software Testing

Types of testing:

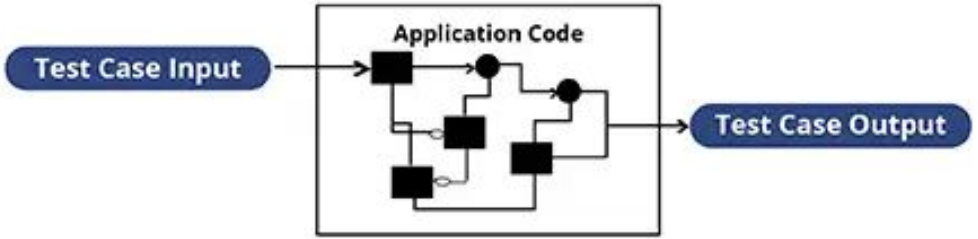
1. White Box Testing
2. Black Box Testing





White Box Testing

- The White Box Test method is the one that looks at the code and structure of the product to be tested and uses that knowledge to perform the tests.
- This method is used in the Unit Testing phase, although it can also occur in other stages such as Integration Tests. For the execution of this method, the tester or the person must have extensive knowledge of the technology used to develop the program.
- White box testing is something that can be used by any organization to test out their applications.
- This type of testing should be a part of any strong software development program. Its always better for an ethical hacker to find a critical vulnerability in your software systems than a malicious hacker who exploits it to gain system or network access.



Black Box Testing

- Black Box Testing is the method that does not consider the internal structure, design, and product implementation to be tested. In other words, the tester does not know its internal functioning.
- The Black Box only evaluates the external behaviour of the system. The inputs received by the system and the outputs or responses it produces are tested.

Black box testing can be done in the following ways:

- **Syntax-Driven Testing –** This type of testing is applied to systems that can be syntactically represented by some language. For example, language can be represented by context-free grammar. In this, the test cases are generated so that each grammar rule is used at least once.
- **Equivalence partitioning –** It is often seen that many types of inputs work similarly, so instead of giving all of them separately, we can group them and test only one input of each group. The idea is to partition the input domain of the system into several equivalence classes, such that each member of the class behaves similarly; i.e., if a test case in one class results in an error, other members of the class will also result in the same error.

Black Box and White Box Testing

Real life scenario

- Suppose there is a registration page in an e-commerce website to be tested. If we are doing black box testing, it will only be checked whether the registration page is working fine or not. But in the case of white box testing, all the functions or classes(which are called) will be checked when that registration page is executed.
- If you are using a calculator, then in the case of black box testing, you will be concerned if the output you are getting is correct or not. But in the case of white box testing, testers will check the internal working of the calculator and how this output was calculated.

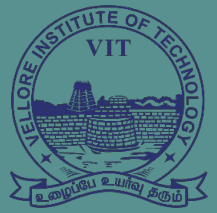
S No.	Black Box Testing	White Box Testing
1	Internal workings of an application are not required.	Knowledge of the internal workings is a must.
2	Also known as closed box/data-driven testing.	Also known as clear box/structural testing.
3	End users, testers, and developers.	Normally done by testers and developers.
4	This can only be done by a trial and error method.	Data domains and internal boundaries can be better tested.

S.no.	On the basis of	Black Box testing	White Box testing
1.	**Basic**	It is a software testing technique that examines the functionality of software without knowing its internal structure or coding.	In white-box testing, the internal structure of the software is known to the tester.
2.	**Also known as**	Black Box Testing is also known as functional testing, data-driven testing, and closed-box testing.	It is also known as structural testing, clear box testing, code-based testing, and transparent testing.
3.	**Programming knowledge**	In black-box testing, there is less programming knowledge is required.	In white-box testing, there is a requirement of programming knowledge.
4.	**Algorithm testing**	It is not well suitable for algorithm testing.	It is well suitable and recommended for algorithm testing.
5.	**Usage**	It is done at higher levels of testing that are system testing and acceptance testing.	It is done at lower levels of testing that are unit testing and integration testing.
6.	**Automation**	It is hard to automate black-box testing due to the dependency of testers and programmers on each other.	It is easy to automate the white box testing.
7.	**Tested by**	It is mainly performed by the software testers.	It is mainly performed by developers.

8.	**Time-consuming**	It is less time-consuming. In Black box testing, time consumption depends upon the availability of the functional specifications.	It is more time-consuming. It takes a long time to design test cases due to lengthy code.
9.	**Base of testing**	The base of this testing is external expectations.	The base of this testing is coding which is responsible for internal working.
10.	**Exhaustive**	It is less exhaustive than White Box testing.	It is more exhaustive than Black Box testing.
11.	**Implementation knowledge**	In black-box testing, there is no implementation knowledge is required.	In white-box testing, there is a requirement of implementation knowledge.
12.	**Aim**	The main objective of implementing black box testing is to specify the business needs or the customer's requirements.	Its main objective is to check the code quality.
13.	**Defect detection**	In black-box testing, defects are identified once the code is ready.	Whereas, in white box testing, there is a possibility of early detection of defects.
14.	**Testing method**	It can be performed by trial and error technique.	It can test data domain and data boundaries in a better way.
15.	**Types**	Mainly, there are three types of black-box testing**: functional testing, Non-Functional testing,** and **Regression testing**.	The types of white box testing are – **Path testing, Loop testing,** and **Condition testing**.
16.	**Errors**	It does not find the errors related to the code.	In white-box testing, there is the detection of hidden errors. It also helps to optimize the code.

Black Box and White Box Testing

Black-box test design techniques- • **Decision table testing** • **All-pairs testing** • **Equivalence partitioning** • **Error guessing**	**White-box test design techniques-** • **Control flow testing** • **Data flow testing** • **Branch testing**
Types of Black Box Testing: • Functional Testing • Non-functional testing • Regression Testing	**Types of White Box Testing:** • Path Testing • Loop Testing • Condition testing



VIT®
BHOPAL
www.vitbhopal.ac.in

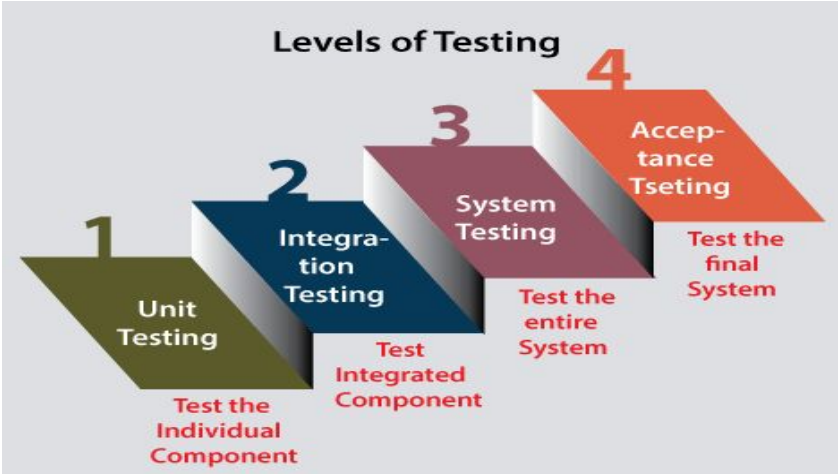
Different Levels of Software Testing

(Unit, Integration, System & Acceptance Testing)

Different Levels of Software Testing

In software testing, we have four different levels of testing, which are as discussed below:

1. **Unit Testing**
2. **Integration Testing**
3. **System Testing**
4. **Acceptance Testing**



Different Levels of Software Testing

1. Unit Testing:

- Unit testing is the first level of software testing, which is used to test if software modules are satisfying the given requirement or not.
- Unit testing involves analysing each unit or an individual component of the software application. A unit component is an individual function or the smallest testable part of the software.
- The primary purpose of executing unit testing is to validate unit components with their performance and test the correctness of inaccessible code.
- Unit testing is also the first level of functional testing.
- Unit testing will help the test engineer and developers in order to understand the base of code that makes them able to change defect causing code quickly. The developers implement the unit.

Different Levels of Software Testing

Integration Testing:

- The second level of software testing is integration testing where individual units are combined and tested as a group.
- The primary purpose of executing the integration testing is to identify the defects at the interaction between integrated components or units.
- When each component or module works separately, we need to check the data flow between the dependent modules, and this process is known as integration testing.
- We only go for the integration testing when the functional testing has been completed successfully on each application module.
- In simple words, we can say that integration testing aims to evaluate the accuracy of communication among all the modules.

Different Levels of Software Testing

System Testing:

- The third level of software testing is system testing, where a complete, integrated system/software is tested, i.e. we will test the application as a whole system.
- The purpose of this test is to evaluate the software's functional and nonfunctional requirements.
- It is end-to-end testing where the testing environment is parallel to the production environment.
- In simple words, we can say that System testing is a sequence of different types of tests to implement and examine the entire working of an integrated software computer system against requirements.

Different Levels of Software Testing

Acceptance Testing:

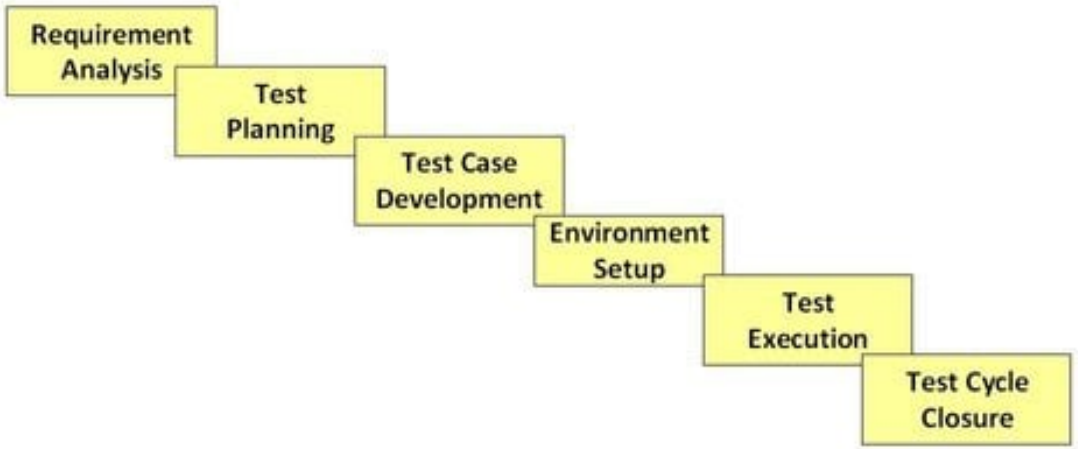
- The last and fourth level of software testing is acceptance testing, where a system is tested for acceptability.
- It is used to evaluate whether a specification or the requirements are met as per its delivery.
- The purpose of this test is to evaluate the system's compliance with the business requirements and assess whether it is acceptable for delivery.
- In simple words, we can say that Acceptance testing is the squeezing of all the testing processes that are previously done.
- The acceptance testing is also known as User acceptance testing (UAT) and is done by the customer before accepting the final product.

VIT®
BHOPAL
www.vitbhopal.ac.in

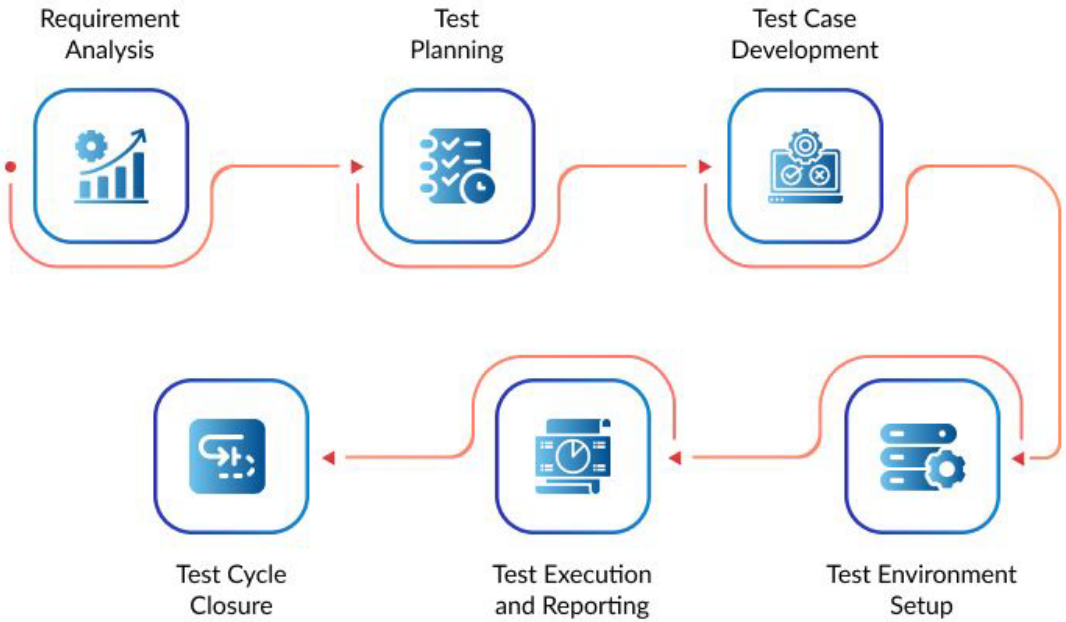
Software Testing Life Cycle (STLC)

What is Software Testing Life Cycle (STLC)?

- Software Testing Life Cycle (STLC) is a sequence of specific activities conducted during the testing process to ensure software quality goals are met.
- STLC involves both verification and validation activities.
- Software Testing is not just a single/isolate activity. It consists of a series of activities carried out methodologically to help certify your software product.



What is Software Testing Life Cycle (STLC)?



Software Testing Life Cycle (STLC) Phases

1. Requirement Phase Testing:

- Identify types of tests to be performed.
- Gather details about testing priorities and focus.
- Prepare Requirement Traceability Matrix (RTM).
- Identify test environment details where testing is supposed to be carried out.

2. Test Planning Activities

- Preparation of test plan/strategy document for various types of testing
- Test tool selection
- Test effort estimation
- Resource planning and determining roles and responsibilities.
- Training requirement

Software Testing Life Cycle (STLC) Phases

3. Test Case Development Activities

- Create test cases, automation scripts (if applicable)
- Review test cases and scripts
- Create test data (If Test Environment is available)

4. Test Environment Setup Activities

- Understand the required architecture and environment set-up
- prepare hardware and software requirement for the Test Environment.
- Setup test Environment and test data
- Perform smoke test on the build

Software Testing Life Cycle (STLC) Phases

5. Test Execution Activities

- Execute tests as per plan
- Document test results, and log defects for failed cases
- Retest the Defect fixes
- Track the defects to closure

6. Test Cycle Closure Activities

- Evaluate cycle completion criteria based on Time, Test coverage, Cost, Software, Critical Business Objectives, Quality
- Prepare test metrics based on the above parameters.
- Document the learning out of the project
- Prepare Test closure report

How to write system test cases?

Writing system test cases involves approximately 10 steps:

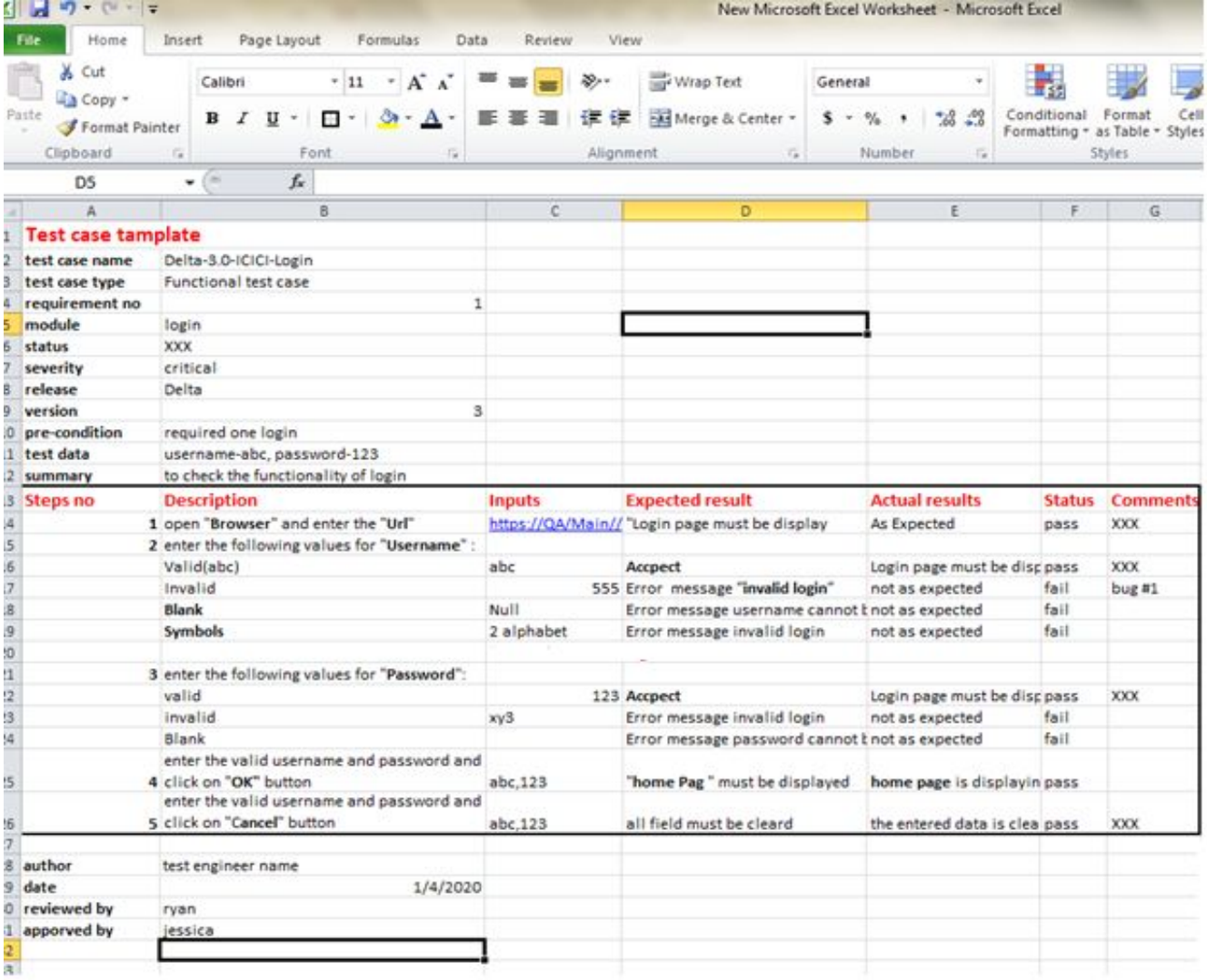
1. **Test case ID:** Generate a unique ID for your test case.
2. **Test case scenario:** Create a one-liner description for it.
3. **Pre-condition:** Your test case must be ready with some predefined conditions. Ex. a registered email id and user name.
4. **Test steps:** Write your test steps with the proper description.
5. **Test data:** Include the data you need for your test cases. Suppose you're testing an application form. You must include test data like the applicant's name, address, contact number, email id, etc.
6. **Expected Result:** You get the predefined test result. Ex. successful submission of an application.
7. **Post conditions:** Some unique data will be generated. Ex. application ID.
8. **Actual result:** As per the expectation.
9. **Status:** Passed or failed.
10. **Review and revise:** Finally, review and revise the test cases as necessary to ensure they are accurate and effective in testing the system requirements.

Sample test case

Test Id	Test Condition	Test Steps	Test Input	Test Expected Result	Actual Result	Status	Remarks
1.	Check that with the correct username and password able to log in.	1. Enter username 2. Enter the password 3. Click on login	username: ABC password: 1234ABa	Login successful	Login successful	Pass	None
2.	Check that if with incorrect username and password able to not login.	1. Enter username 2. Enter password 3. Click on login	username: AB1 password: 1234AB	Login unsuccessful	Login unsuccessful	Pass	None
3.	Check if the loading page loading efficiently for the client.	1. Click on login button.	None	Welcome to login page.	Welcome to login page.	Fail	The login page is not loaded due to a browser compatibility issue on the user's side.

Example of a Test Case Template

Here, we are writing a test case for the **ICICI application's Login** module:



	A	B	C	D	E	F	G
1	**Test case tamplate**						
2	**test case name**	Delta-3.0-ICICI-Login					
3	**test case type**	Functional test case					
4	**requirement no**	1					
5	**module**	login					
6	**status**	XXX					
7	**severity**	critical					
8	**release**	Delta					
9	**version**	3					
10	**pre-condition**	required one login					
11	**test data**	username-abc, password-123					
12	**summary**	to check the functionality of login					
13	**Steps no**	**Description**	**Inputs**	**Expected result**	**Actual results**	**Status**	**Comments**
14	1	open "**Browser**" and enter the "**Url**"	https://QA/Main//	"Login page must be display	As Expected	pass	XXX
15	2	enter the following values for "**Username**" :					
16		Valid(abc)	abc	**Accpect**	Login page must be disp	pass	XXX
17		Invalid	555	Error message "**invalid login**"	not as expected	fail	bug #1
18		**Blank**	Null	Error message username cannot b	not as expected	fail	
19		**Symbols**	2 alphabet	Error message invalid login	not as expected	fail	
20							
21	3	enter the following values for "**Password**":					
22		valid	123	**Accpect**	Login page must be disp	pass	XXX
23		invalid	xy3	Error message invalid login	not as expected	fail	
24		Blank		Error message password cannot b	not as expected	fail	
25	4	enter the valid username and password and click on "**OK**" button	abc,123	"**home Pag** " must be displayed	**home page** is displayin	pass	
26	5	enter the valid username and password and click on "**Cancel**" button	abc,123	all field must be cleard	the entered data is clea	pass	XXX
27							
28	**author**	test engineer name					
29	**date**	1/4/2020					
30	**reviewed by**	ryan					
31	**apporved by**	jessica					
32							

System Testing Metrics

- **A software testing metric indicates the degree to which a process, component, or tool is efficient.**

Importance of Metrics in Software Testing

- Software testing metrics are used to increase the overall productivity of the development process.
- It helps to make more informed choices about the tools and technologies being used.
- It helps to identify unique ways and techniques that are beneficial for their system, hence increasing performance.
- Software testing metrics determine the health of a process, tool, and approach used.

Types of System Testing Metrics

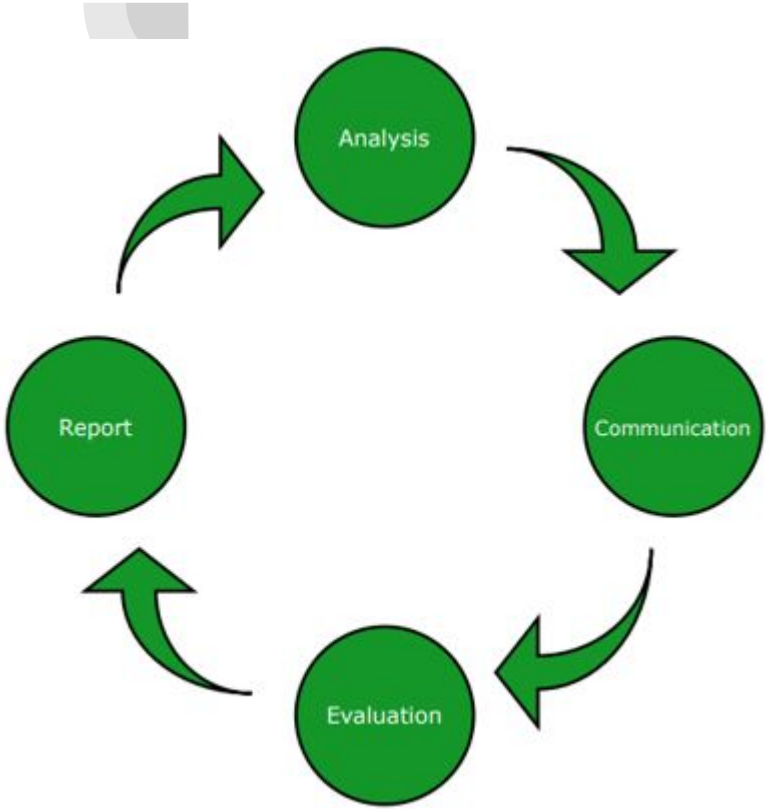
There are three types of software testing metrics:

1. **Process Metrics:** The quantitative measures that define the efficiency of a process based on parameters like speed, time, utilization of resources, etc.
2. **Product Metrics:** Measures to determine the quality, size, performance, and efficiency of the product come under product metrics.
3. **Project Metrics:** Quality and productivity of the project, utilization of resources, cost, and time come under the project metrics.

Manual Test Metrics: Since manual testing is a step-by-step testing process and they are little different. There are two types of manual test metrics:

- **Base Metrics:** The essential data, taken during the testing process, comes under base metrics. It comprises test cases and test cases completed.

Metrics Life Cycle



Analysis

- Identification of the Metrics
- Define the identified Metrics.

Communicate

- Explain the need of metric to stakeholder and testing team
- Educate the testing team about the data points need to be captured for processing the metric.

Evaluation

- Capture & verify the data.
- Calculating the metric(s) value using the data captured

Report

- Develop the report with effective conclusion
- Distribute report to the stakeholder and respective representative.
- Take feedback from stakeholder.

- **Example of Software Test Metrics Calculation**

S No.	Testing Metric	Data retrieved during test case development
1	No. of requirements	5
2	The average number of test cases written per requirement	40
3	Total no. of Test cases written for all requirements	200
4	Total no. of Test cases executed	164
5	No. of Test cases unexecuted	36
6	No. of Test cases passed	100
7	No. of Test cases failed	60
8	No. of Test cases blocked	4
9	Total no. of defects identified	20
10	Defects accepted as valid by the dev team	15
11	Defects deferred for future releases	5
12	Defects fixed	12

Percentage test cases executed

Test Case Effectiveness

Failed Test Cases Percentage

Blocked Test Cases Percentage

Fixed Defects Percentage

Accepted Defects Percentage

Defects Deferred Percentage

- **Example of Software Test Metrics Calculation**

S No.	Testing Metric	Data retrieved during test case development
1	No. of requirements	5
2	The average number of test cases written per requirement	40
3	Total no. of Test cases written for all requirements	200
4	Total no. of Test cases executed	164
5	No. of Test cases unexecuted	36
6	No. of Test cases passed	100
7	No. of Test cases failed	60
8	No. of Test cases blocked	4
9	Total no. of defects identified	20
10	Defects accepted as valid by the dev team	15
11	Defects deferred for future releases	5
12	Defects fixed	12

1. Percentage test cases executed =
(No of test cases executed / Total no of test cases written) x 100
= (164 / 200) x 100
= 82

2. Test Case Effectiveness =
(Number of defects detected / Number of test cases run) x 100
= (20 / 164) x 100
= 12.2

3. Failed Test Cases Percentage = *(Total number of failed test cases / Total number of tests executed) x 100*
*= (60 / 164) * 100*
= 36.59

4. Blocked Test Cases Percentage = *(Total number of blocked tests / Total number of tests executed) x 100*
*= (4 / 164) * 100*
= 2.44

- **Example of Software Test Metrics Calculation**

S No.	Testing Metric	Data retrieved during test case development
1	No. of requirements	5
2	The average number of test cases written per requirement	40
3	Total no. of Test cases written for all requirements	200
4	Total no. of Test cases executed	164
5	No. of Test cases unexecuted	36
6	No. of Test cases passed	100
7	No. of Test cases failed	60
8	No. of Test cases blocked	4
9	Total no. of defects identified	20
10	Defects accepted as valid by the dev team	15
11	Defects deferred for future releases	5
12	Defects fixed	12

5. Fixed Defects Percentage = *(Total number of flaws fixed / Number of defects reported) x 100*

*= (12 / 20) * 100*

= 60

6. Accepted Defects Percentage = *(Defects Accepted as Valid by Dev Team / Total Defects Reported) x 100*

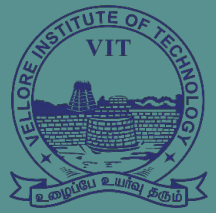
*= (15 / 20) * 100*

= 75

7. Defects Deferred Percentage = *(Defects deferred for future releases / Total Defects Reported) x 100*

*= (5 / 20) * 100*

= 25



VIT®
BHOPAL
www.vitbhopal.ac.in

Software Testing Tools

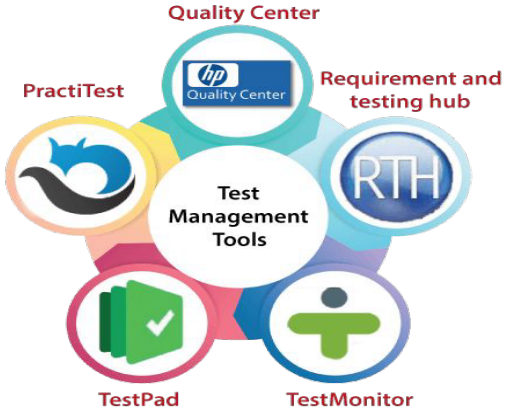
System Testing Tools

- The software testing tools can be categorized, depending on the licensing (paid or commercial, open-source), technology usage, type of testing, and so on.
- With the help of testing tools, we can improve our software performance, deliver a high-quality product, and reduce the duration of testing, which is spent on manual efforts.
- The software testing tools can be divided into the following:
    - **Test management tool**
    - **Bug tracking tool**
    - **Automated testing tool**
    - **Performance testing tool**
    - **Cross-browser testing tool**
    - **Integration testing tool**
    - **Unit testing tool**
    - **Mobile/android testing tool**
    - **GUI testing tool**
    - **Security testing tool**

System Testing Tools

1. Test management tool

Test management tools are used to keep track of all the testing activity, fast data analysis, manage manual and automation test cases, various environments, and plan and maintain manual testing as well.



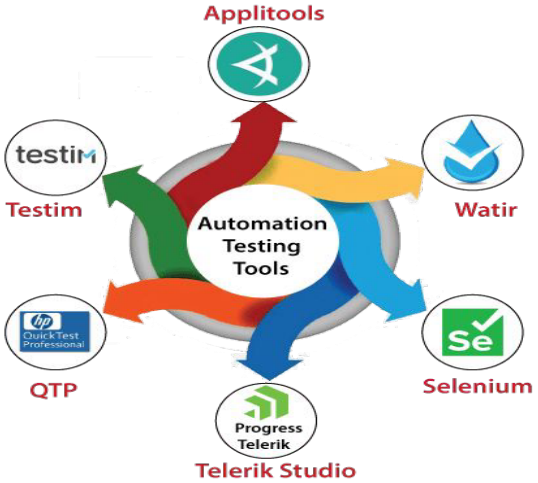
2. Bug/ defect tracking tool

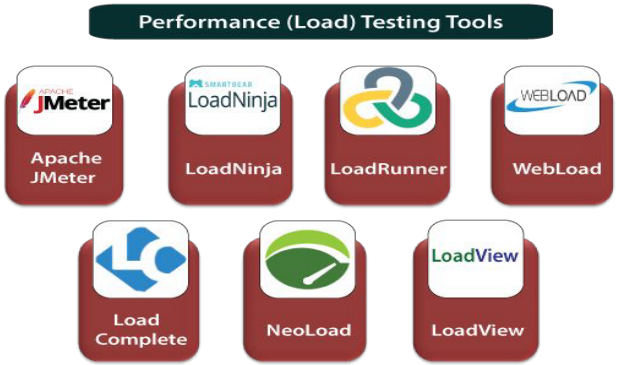
The defect tracking tool is used to keep track of the bug fixes and ensure the delivery of a quality product. This tool can help us to find the bugs in the testing stage so that we can get the defect-free data in the production server.

System Testing Tools

3. Automation testing tool

This type of tool is used to enhance the productivity of the product and improve the accuracy. We can reduce the time and cost of the application by writing some test scripts in any programming language.





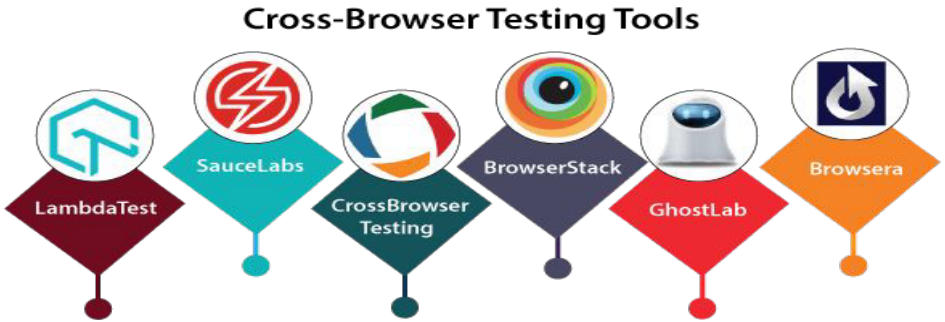
4. Performance testing tool

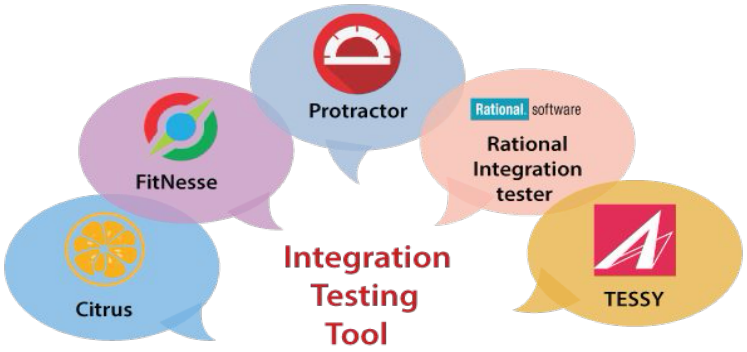
Performance or Load testing tools are used to check the load, stability, and scalability of the application. When n-number of the users are using the application at the same time, and if the application gets crashed because of the immense load, to get through this type of issue, we need load testing tools.

System Testing Tools

5. Cross-browser testing tool

This type of tool is used when we need to compare a web application in the various web browser platforms. It will ensure the consistent behaviour of the application in multiple devices, browsers, and platforms.





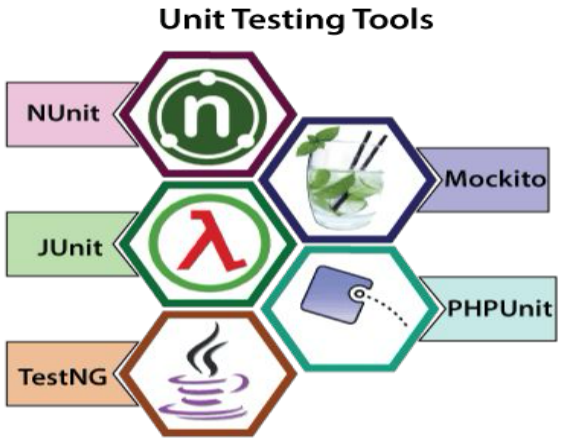
6. Integration testing tool

This type of tool is used to test the interface between modules and find the critical bugs that are happening because of the different modules and ensuring that all the modules are working as per the client requirements.

System Testing Tools

7. Unit testing tool

This testing tool is used to help the programmers to improve their code quality, and with the help of these tools, they can reduce the time of code and the overall cost of the software.



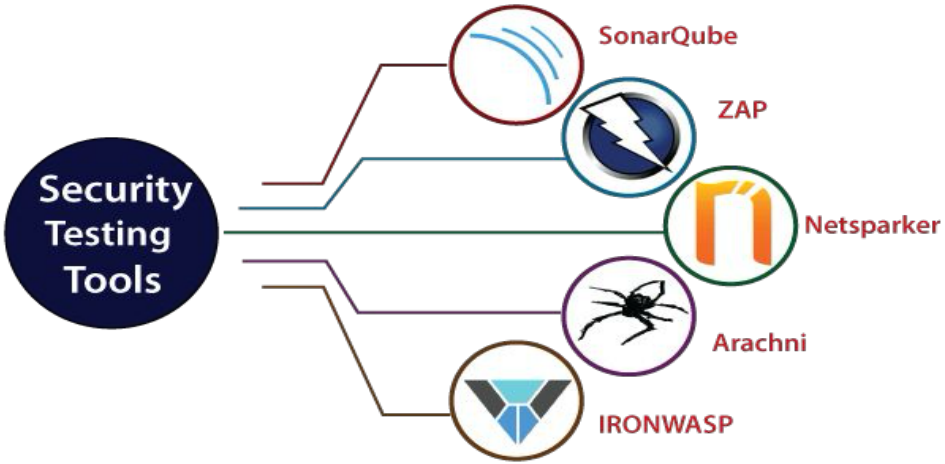
8. Mobile/android testing tool

We can use this type of tool when we are testing any mobile application. Some of the tools are open-source, and some of the tools are licensed. Each tool has its functionality and features.

System Testing Tools

9. GUI testing tool

A GUI testing tool is used to test the User interface of the application. These types of tools will help to grab the user's attention and find the loopholes in the application's design and make it better.



10. Security testing tool

The security testing tool is used to ensure the security of the software. If any security loophole is there, it could be fixed at the early stage of the product. We need this type of the tool when the software has encoded the security code which is not accessible by the unauthorized users.

Advantages and Disadvantages of System Testing

The advantages are-

- It checks the overall functionality of a system to fulfil all the business requirements.
- As it checks the whole system, it easily detects the bugs and finds defects skipped in unit and integration testing. Thus it makes the product ready for acceptance testing.
- You can perform the testing in a practical environment. Thus it helps to detect real-time bugs.

The disadvantages are-

- This testing is time-consuming and expensive because it checks the full system.
- System testing becomes challenging for large and complex systems.

Testing Vs Debugging

#	Testing	Debugging
1	Starts with known conditions. Uses predefined procedure. Has predictable outcomes.	Starts with possibly unknown initial conditions. End cannot be predicted.
2	Planned, Designed and Scheduled.	Procedures & Duration are not constrained.
3	A demo of an error or apparent correctness.	A Deductive process.
4	Proves programmer's success or failure.	It is programmer's Vindication.
5	Should be predictable, dull, constrained, rigid & inhuman.	There are intuitive leaps, conjectures, experimentation & freedom.
6	Much of testing can be without design knowledge.	Impossible without a detailed design knowledge.
7	Can be done by outsider to the development team.	Must be done by an insider (development team).
8	A theory establishes what testing can do or cannot do.	There are only Rudimentary Results (on how much can be done. Time, effort, how etc. depends on human ability).
9	Test execution and design can be automated.	Debugging - Automation is a dream.